InternNova Internship Program

Week 4 Assignment

Cyber Security – 6 Week Batch

Vulnerability Assessment & Penetration Testing

Intern Name	NAJMUDEEN S
Internship	InternNova
Week	4
Assignment	Vulnerability Assessment & Penetration Testing
Month	October 2026

Cyber Security Internship Report

Contents

1 Task 1: Understanding Vulnerabilities **1**

1.1 Introduction . . . 1
1.2 What is a Vulnerability? . . . 1
1.2.1 Definition . . . 1
1.2.2 Real-World Example . . . 1
1.2.3 Cybersecurity Connection . . . 1
1.3 What is a Threat? . . . 1
1.3.1 Definition . . . 1
1.3.2 Real-World Example . . . 2
1.3.3 Cybersecurity Connection . . . 2
1.4 What is Risk? . . . 2
1.4.1 Definition . . . 2
1.4.2 Real-World Example . . . 2
1.4.3 Cybersecurity Connection . . . 2
1.5 What is an Exploit? . . . 2
1.5.1 Definition . . . 2
1.5.2 Real-World Example . . . 2
1.5.3 Cybersecurity Connection . . . 2
1.6 What is Vulnerability Assessment? . . . 3
1.6.1 Definition . . . 3
1.6.2 Expected Outcome . . . 3
1.7 Vulnerability vs. Threat vs. Risk . . . 3
1.8 Quick Check . . . 5
1.9 Task 1 Summary . . . 7

2 Task 2: Common Security Vulnerabilities **8**

2.1 SQL Injection (SQLi) . . . 8
2.1.1 Definition . . . 8
2.1.2 Effect on System . . . 8
2.1.3 Possible Impact . . . 8
2.1.4 Basic Prevention . . . 8
2.2 Cross-Site Scripting (XSS) . . . 9
2.2.1 Definition . . . 9
2.2.2 Effect on System . . . 9
2.2.3 Possible Impact . . . 10
2.2.4 Basic Prevention . . . 10
2.3 Broken Authentication . . . 10
2.3.1 Definition . . . 11
2.3.2 Effect on System . . . 11
2.3.3 Possible Impact . . . 11
2.3.4 Basic Prevention . . . 11
2.4 Security Misconfiguration . . . 11
2.4.1 Definition . . . 13
2.4.2 Effect on System . . . 13
2.4.3 Possible Impact . . . 13
2.4.4 Basic Prevention . . . 13
2.5 Vulnerable and Outdated Components . . . 13
2.5.1 Definition . . . 14

2.5.2 Effect on System . . . 14
2.5.3 Possible Impact . . . 14
2.5.4 Basic Prevention . . . 14
2.6 Comparison of the Five Vulnerabilities . . . 15
2.7 Quick Check . . . 15
2.8 Mini Practical – Vulnerability-to-Control Mapping . . . 15
2.9 Task 2 Summary . . . 18

3 Task 3: Vulnerability Assessment Process 19
3.1 Introduction . . . 19
3.2 Step 1 – Identify . . . 19
3.2.1 What Happens . . . 19
3.2.2 Why It Is Important . . . 19
3.2.3 Expected Outcome . . . 19
3.3 Step 2 – Scan . . . 19
3.3.1 What Happens . . . 19
3.3.2 Why It Is Important . . . 20
3.3.3 Expected Outcome . . . 20
3.4 Step 3 – Analyze . . . 20
3.4.1 What Happens . . . 20
3.4.2 Why It Is Important . . . 20
3.4.3 Expected Outcome . . . 20
3.5 Step 4 – Prioritize . . . 20
3.5.1 What Happens . . . 20
3.5.2 Why It Is Important . . . 20
3.5.3 Expected Outcome . . . 21
3.6 Step 5 – Remediate . . . 21
3.6.1 What Happens . . . 21
3.6.2 Why It Is Important . . . 21
3.6.3 Expected Outcome . . . 21
3.7 Step 6 – Re-scan and Verify . . . 21
3.7.1 What Happens . . . 21
3.7.2 Why It Is Important . . . 21
3.7.3 Expected Outcome . . . 21
3.8 Quick Check . . . 23
3.9 Task 3 Summary . . . 25

4 Task 4: Introduction to Penetration Testing 26
4.1 Scope, Rules of Engagement and Authorization . . . 26
4.1.1 Scope . . . 26
4.1.2 Out-of-Scope Systems . . . 26
4.1.3 Testing Constraints . . . 26
4.2 Why Written Permission Is Important . . . 26
4.2.1 Authorization Details . . . 26
4.2.2 Emergency Contact . . . 26
4.3 VA and PT – Practical Difference . . . 26
4.3.1 Coverage . . . 26
4.3.2 Output . . . 26
4.4 Ethical and Professional Considerations . . . 27
4.5 What is Penetration Testing? . . . 27

4.5.1 Definition . . . 27
4.5.2 Why Penetration Testing Is Performed . . . 27
4.6 Vulnerability Assessment vs. Penetration Testing . . . 27
4.7 Authorized Security Testing . . . 29
4.7.1 Written Permission . . . 29
4.7.2 Why Authorization Matters . . . 29
4.8 Penetration Testing Lifecycle . . . 29
4.8.1 Planning . . . 29
4.8.2 Reconnaissance . . . 29
4.8.3 Scanning . . . 29
4.8.4 Vulnerability Analysis . . . 29
4.8.5 Security Testing . . . 29
4.8.6 Reporting . . . 30
4.9 Authorized Penetration Testing Environment . . . 30
4.10 Quick Check . . . 30
4.11 Task 4 Summary . . . 33

5 Task 5: Safe Practical Activity 34
5.1 Introduction . . . 34
5.2 Tool Used – Nessus Essentials . . . 34
5.3 Practical Objective . . . 34
5.4 Why Nessus Was Selected . . . 35
5.4.1 Assessment Workflow . . . 35
5.5 Nessus Assessment Observations . . . 35
5.5.1 Severity Distribution . . . 35
5.5.2 Plugin-Level Evidence . . . 37
5.6 Observed Vulnerability Results . . . 37
5.7 Finding 1 – UnrealIRCd Backdoor Detection . . . 37
5.7.1 Finding . . . 37
5.7.2 Risk Level . . . 38
5.7.3 Potential Impact . . . 38
5.7.4 Recommendation . . . 38
5.8 Finding 2 – Apache Tomcat AJP Connector Request Injection . . . 38
5.8.1 Finding . . . 38
5.8.2 Risk Level . . . 38
5.8.3 Potential Impact . . . 39
5.8.4 Recommendation . . . 39
5.9 Security and Safety Rule . . . 39
5.10 Quick Check . . . 40
5.11 Task 5 Summary . . . 42

6 Task 6: Security Finding Report 43
6.1 Introduction . . . 43
6.2 Finding 1 – UnrealIRCd Backdoor Detection . . . 43
6.3 Finding 2 – Apache Tomcat AJP Connector Request Injection . . . 44
6.4 How to Read a Nessus Finding . . . 44
6.4.1 CVSS . . . 44
6.4.2 VPR . . . 44
6.4.3 EPSS . . . 44
6.5 Finding Documentation Quality . . . 44

6.5.1 Finding . . . 45
6.5.2 Risk Level . . . 45
6.5.3 Potential Impact . . . 45
6.5.4 Recommendation . . . 45
6.6 Remediation Priority . . . 45
6.7 Vulnerability Finding and Risk Prioritization . . . 45
6.8 Security Risk Rating . . . 45
6.9 Task 6 Summary . . . 48
6.10 Final Week 4 Summary . . . 48

7 VAPT Assessment Checklist and Reporting Notes 49
7.1 Evidence Handling Notes . . . 49
7.2 Professional Reporting Language . . . 49
7.3 Final Learning Reflection . . . 49

8 Appendix A: Vulnerability Assessment Working Notes 50
8.1 Asset Review Questions . . . 50
8.2 Finding Review Questions . . . 50
8.3 Severity Interpretation . . . 50
8.4 False Positive Review . . . 50
8.5 False Negative Awareness . . . 50

9 Appendix B: Professional VAPT Report Checklist 51
9.1 Evidence Presentation Principles . . . 51
9.2 Writing Principles . . . 51
9.3 Closing Reflection . . . 51

10 Appendix C: VAPT Terminology and Practical Notes 52
10.1 Why Consistent Terminology Matters . . . 52

11 Appendix D: Vulnerability Assessment Review Questions 53
11.1 Foundational Concepts . . . 53
11.2 Common Vulnerabilities . . . 53
11.3 Assessment Process . . . 53
11.4 Penetration Testing . . . 53
11.5 Nessus Practical . . . 53

12 Appendix E: Final VAPT Workflow Reference 54
12.1 Professional Skills Demonstrated . . . 54
12.2 Connection to SOC and Security Operations . . . 54
12.3 Connection to Future Penetration Testing . . . 54
12.4 Final Reflection . . . 54

13 Appendix F: Quick Reference for Future Assessments 55
13.1 Final Checklist . . . 55

1 Task 1: Understanding Vulnerabilities

1.1 Introduction

A vulnerability is a weakness in a system, application, network, configuration, process, or security control that could be used to cause an unwanted security outcome. Vulnerabilities can arise from insecure design, implementation errors, weak configurations, outdated software, missing security controls, or poor operational practices.

Understanding the relationship between vulnerabilities, threats, exploits, impacts, and risk is essential before performing vulnerability assessment or penetration testing. A vulnerability by itself does not always mean that an attack will occur; risk depends on factors such as the likelihood of exploitation and the potential impact.

1.2 What is a Vulnerability?

1.2.1 Definition

A vulnerability is a weakness, flaw, or condition in a system, application, network, configuration, process, or security control that can reduce the security of an asset. The weakness may exist because of insecure design, programming errors, incorrect configuration, outdated software, weak authentication, excessive privileges, or missing security controls. A vulnerability describes a condition that creates an opportunity for an unwanted security outcome; it does not by itself prove that an attack has occurred.

From a vulnerability-management perspective, weaknesses should be identified, validated, documented, prioritized, and addressed according to their technical and business context. The same weakness may require different levels of attention depending on the affected asset, exposure, available controls, and potential impact.

1.2.2 Real-World Example

A web application using an outdated third-party library with a publicly known security weakness contains a vulnerability. The weakness may remain present even if no attacker is currently attempting to use it.

1.2.3 Cybersecurity Connection

Vulnerability management focuses on discovering, validating, prioritizing, and reducing weaknesses before they can become significant security incidents.

1.3 What is a Threat?

1.3.1 Definition

A threat is a potential source, event, or circumstance that could cause harm to an asset. In cybersecurity, a threat can involve a malicious actor, an accidental action, a technical failure, or an environmental event. The important point is that a threat represents the possibility of harm, while a vulnerability represents a weakness that may contribute to that harm.

For example, an attacker attempting to compromise an exposed server is a threat. The threat becomes more significant when the target contains exploitable weaknesses, has valuable information, or is insufficiently protected by security controls. Threat awareness therefore helps security teams understand where vulnerabilities may be most relevant.

1.3.2 Real-World Example

An attacker attempting to compromise an exposed server is a threat to the organization operating that server.

1.3.3 Cybersecurity Connection

Threat information helps security teams understand which vulnerabilities and assets deserve greater attention.

1.4 What is Risk?

1.4.1 Definition

Risk represents the potential for loss or harm when a threat can take advantage of a vulnerability. Risk is therefore broader than the vulnerability itself. It considers factors such as the likelihood that an unwanted event could occur and the potential consequences if it does occur.

In practical cybersecurity work, risk is influenced by technical severity, exploitability, exposure, asset criticality, existing security controls, and business impact. This is why security teams do not always remediate every finding in exactly the same order. Risk-based prioritization helps direct limited resources toward weaknesses that could create the greatest overall concern.

1.4.2 Real-World Example

If a critical server has a remotely exploitable vulnerability and contains sensitive information, the likelihood and impact of exploitation may make the associated risk high.

1.4.3 Cybersecurity Connection

Risk-based prioritization allows security teams to address weaknesses according to their potential business and technical consequences rather than treating every finding equally.

1.5 What is an Exploit?

1.5.1 Definition

An exploit is a method, technique, or code that takes advantage of a vulnerability to produce an unintended security effect. An exploit may involve a specially crafted request, malicious input, a sequence of actions, or another technique designed to trigger the weakness.

The existence of an exploit does not automatically mean that every vulnerable system can be compromised in the same way. Exploitability depends on the affected software, configuration, privileges, network exposure, required conditions, and other security controls. In professional penetration testing, exploitation is performed only when it is explicitly authorized and appropriate for the defined scope.

1.5.2 Real-World Example

A crafted request that abuses a vulnerable application component to read unauthorized files is an example of exploitation.

1.5.3 Cybersecurity Connection

Penetration testing may validate exploitability in a controlled and explicitly authorized environment. Exploitation outside an approved scope is not permitted.

1.6 What is Vulnerability Assessment?

1.6.1 Definition

Vulnerability assessment is a systematic process of identifying, analyzing, prioritizing, and reporting security weaknesses in authorized systems. It may use automated scanners, configuration reviews, manual analysis, security advisories, and other assessment techniques to build an understanding of the security condition of the target.

The purpose is not simply to generate a long list of vulnerabilities. A useful assessment connects findings with affected assets, severity, evidence, potential impact, and remediation recommendations. The results can then support remediation planning and later verification through re-scanning or re-assessment.

1.6.2 Expected Outcome

The expected outcome is a structured list of findings with severity or risk information and remediation recommendations. Re-scanning can then be used to verify whether identified weaknesses have been addressed.

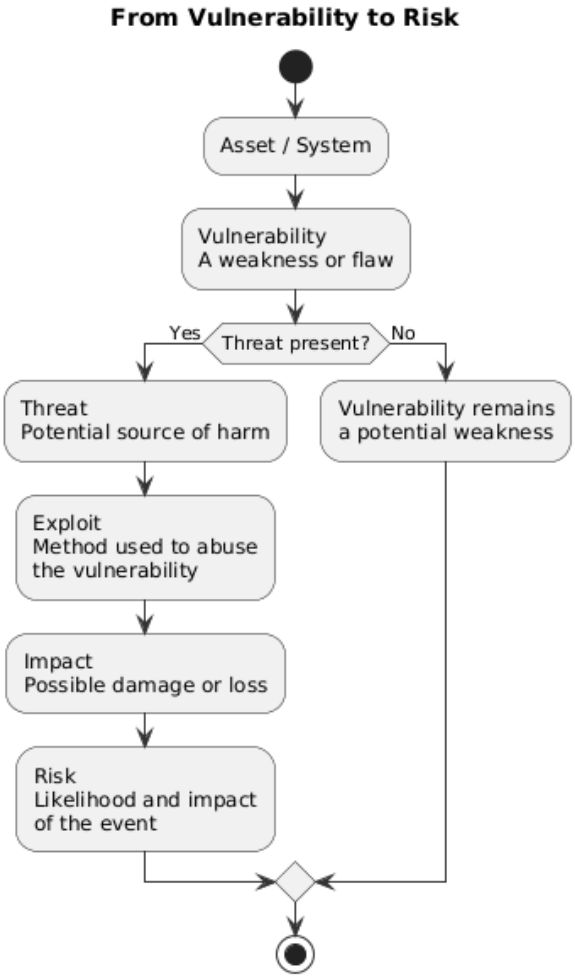


Figure 1: Relationship between vulnerability, threat, exploit, impact and risk

1.7 Vulnerability vs. Threat vs. Risk

subsectionSecurity Context of the Four Concepts The four concepts should be understood as connected but distinct parts of security analysis. A vulnerability describes a weakness, a threat describes a potential source of harm, an exploit describes a technique that may take advantage of the weakness,

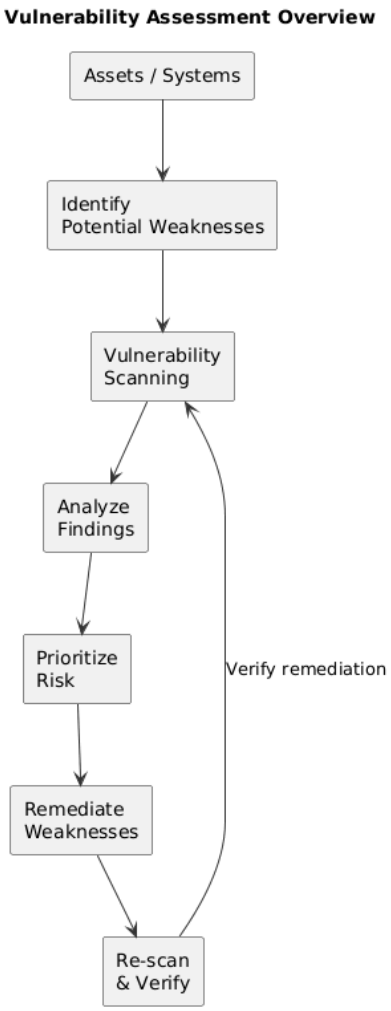


Figure 2: Overview of the vulnerability assessment process

and risk describes the potential consequence when those conditions interact. Keeping these terms separate helps security teams communicate findings accurately and prevents technical observations from being exaggerated.

subsubsectionWhy the Distinction Matters In a professional assessment, saying that a system is "vulnerable" does not automatically mean that it has been compromised. Similarly, identifying a threat does not prove that the threat actor has access to the system. Security reporting becomes more useful when each statement is supported by evidence and the assessor clearly distinguishes between an observed condition, a potential scenario, and a confirmed security event.

subsubsectionFrom Technical Finding to Security Risk A scanner may identify an outdated service version, but the final priority depends on additional context. The assessor should consider whether the service is reachable, whether exploitation is known or plausible, what privileges are available, what information is protected, and whether compensating controls reduce exposure. This process converts a technical finding into a meaningful risk assessment.

subsectionAsset Perspective Every vulnerability exists in relation to an asset. The asset may be a server, workstation, application, database, network device, cloud resource, identity, or information repository. Identifying the asset is important because the value and role of the asset influence the consequences of a security event.

subsubsectionAsset Criticality A vulnerability affecting a public-facing authentication server may deserve more immediate attention than the same weakness on an isolated training machine. Asset criticality therefore provides an important context for prioritization.

subsubsectionExposure and Reachability Exposure describes how easily a potential threat can interact with the affected asset. Internet-facing services generally have a broader attack surface than systems restricted to a controlled internal network. Reachability should therefore be considered when reviewing scanner findings and deciding remediation priority.

Concept	Meaning	Simple Example
Vulnerability	A weakness that can be abused or cause a security problem.	An outdated server component with a known security flaw.
Threat	A potential source of harm.	An attacker attempting to compromise an exposed server.
Risk	Potential loss or harm considering likelihood and impact.	A critical Internet-facing weakness on a sensitive server.
Exploit	A method used to take advantage of a vulnerability.	A crafted request that triggers a known vulnerable component.

Table 1: Comparison of vulnerability, threat, risk and exploit

1.8 Quick Check

1. What is the difference between a vulnerability and a threat?
2. Why does a vulnerability not automatically mean that a security incident will occur?
3. What factors commonly influence risk?
4. What is an exploit?
5. What is the purpose of vulnerability assessment?

subsectionPractical Interpretation of Risk A security analyst should avoid treating a vulnerability list as a complete risk register. The analyst must connect each technical observation to the affected asset,

exposure, likelihood, and impact. This is particularly important when multiple vulnerabilities exist on the same host because the combined attack surface may be more significant than any individual finding.

subsubsectionAnalyst Mindset The analyst should ask four questions: What is weak? Who could take advantage of it? What could happen if it were abused? What control would reduce the exposure? These questions form a simple reasoning framework for reviewing both automated findings and manually identified weaknesses.

newpage
subsectionExtended Technical Analysis: Understanding Vulnerability Context A vulnerability should be understood as a condition within a larger security environment rather than as an isolated technical label. In a real assessment, the analyst first identifies the asset, then determines what weakness exists, how the weakness can be reached, and what security controls surround it. This approach prevents a common reporting mistake in which a scanner result is copied directly into a report without considering the system's actual context.

subsubsectionRelationship Between Asset and Vulnerability The value of a vulnerability is strongly influenced by the asset it affects. A weakness on a disposable laboratory machine may have limited business significance, while the same weakness on an authentication server, database, or public-facing application can have much greater consequences. Asset ownership and business function should therefore be recorded as part of assessment preparation.

subsubsectionRelationship Between Threat and Vulnerability A vulnerability provides an opportunity, but a threat represents the potential source of harm. Security teams use threat information to determine which weaknesses are more likely to be relevant. For example, an Internet-facing service may receive more attention than an isolated service because it is reachable by a much broader population.

subsubsectionRelationship Between Exploit and Impact An exploit is the technique used to abuse a weakness. The existence of an exploit can increase concern because it demonstrates that the weakness may be practically usable. However, the resulting impact still depends on the privileges available, the information stored, the network position, and other controls.

subsubsectionRisk-Based Decision Making Risk-based decision making connects technical assessment with operational priorities. Security teams cannot always correct every issue simultaneously. They therefore use severity, likelihood, exploitability, exposure, asset importance, and impact to decide what should be handled first.

subsubsectionExample Security Scenario Consider an application server that exposes an outdated component. The outdated component represents a vulnerability. A malicious actor attempting to reach the service represents a threat. A technique that takes advantage of the vulnerable component represents an exploit. If successful access could expose sensitive information or allow unauthorized actions, the potential consequence contributes to risk. The security team's response should therefore address both the technical weakness and the circumstances that make exploitation meaningful.

subsectionAnalyst Questions for a New Finding When reviewing a new vulnerability, the following questions provide a repeatable analytical method:
beginenumerate
item What asset is affected, and who owns it?
item Is the affected service reachable from an untrusted network?
item What component or configuration creates the weakness?
item Is there reliable evidence that the finding applies to the target?
item Are known exploitation techniques available?
item What privileges could an attacker obtain if the weakness were abused?
item What information or services could be affected?

item Are compensating controls already reducing the exposure?
item What remediation is practical and what verification will be required?
endenumerate

subsectionTask 1 Learning Outcome The main outcome of Task 1 is the ability to describe security weaknesses using precise terminology. Instead of treating every vulnerability as an attack, the analyst learns to separate the weakness, the potential threat, the exploitation method, the resulting impact, and the overall risk. This vocabulary becomes important in later tasks because vulnerability scanners and penetration-testing reports rely on the same distinctions.

newpage
subsectionCase Study: From Weakness to Risk Consider a hypothetical web server used to host an internal business application. During an assessment, the assessor discovers an outdated software component. The first observation is the vulnerability itself: the deployed component contains a known security weakness. The assessor then asks whether the service is reachable, whether the vulnerable feature is enabled, and whether the server contains sensitive information.

If the service is reachable from an untrusted network and a reliable exploitation technique exists, the likelihood of exploitation may be higher. If the server stores sensitive business information or provides an important business function, the potential impact may also be high. The resulting risk is therefore influenced by more than the vulnerability name alone.

This example demonstrates why vulnerability, threat, exploit, impact, and risk should be documented separately. It also shows why vulnerability management is closely connected with asset management and business context.

subsectionPractical Analyst Exercise For a new finding, an analyst can create a short reasoning record:
beginenumerate
item Identify the affected asset.
item Identify the exact weakness.
item Identify the exposed service or application function.
item Determine whether exploitation conditions are present.
item Identify the possible security impact.
item Review existing controls.
item Assign a remediation priority.
item Define how the correction will be verified.
endenumerate

This exercise helps prevent the common mistake of copying a scanner's severity directly into a final risk statement. The analyst's role is to interpret the evidence and communicate a reasoned security decision.

1.9 Task 1 Summary

Vulnerabilities are weaknesses, threats are potential sources of harm, exploits are methods used to abuse weaknesses, and risk represents potential loss or harm. These concepts form the foundation for vulnerability assessment and penetration testing.

2 Task 2: Common Security Vulnerabilities

2.1 SQL Injection (SQLi)

subsubsectionHow SQL Injection Occurs SQL Injection generally occurs when an application constructs database queries by directly combining trusted query syntax with untrusted user input. If the application does not maintain a clear separation between commands and data, specially crafted input can alter the intended meaning of the query.

The weakness can appear in login forms, search functions, filtering parameters, URL parameters, APIs, or administrative interfaces that interact with a database. The important security boundary is the point where externally influenced data enters the database query.

subsubsectionSecurity Analysis The impact of SQL Injection depends heavily on the privileges of the database account used by the application. A highly privileged database account can make a database-layer weakness considerably more serious. Applying least privilege at the database layer therefore provides defense in depth even when an application-level flaw exists.

subsubsectionSecure Development Perspective Prepared statements and parameterized queries are the primary control because they separate the SQL command structure from user-supplied values. Input validation can provide an additional layer of defense, but it should not be treated as a replacement for safe query construction.

2.1.1 Definition

SQL Injection occurs when untrusted input is incorporated into SQL statements without appropriate validation or parameterization. An attacker may manipulate the intended database query. In practice, the weakness occurs when application input crosses the boundary between ordinary data and database commands without adequate separation. Parameterized queries are important because they keep supplied values separate from the SQL statement structure.

2.1.2 Effect on System

A vulnerable application may process attacker-controlled input as part of a database command rather than as ordinary data.

2.1.3 Possible Impact

Potential impacts include unauthorized data access, data modification, authentication bypass, and in some situations broader compromise depending on the application's privileges and database configuration.

2.1.4 Basic Prevention

Use parameterized queries or prepared statements, validate input appropriately, apply least-privilege database permissions, and avoid constructing SQL statements through unsafe string concatenation.

Reference Basis: OWASP Top 10 – Injection [View Reference]

Reference / Standard: OWASP Web Security Testing Guide – SQL Injection Testing [View Reference]

Learning Video: Professor Messer – Web and Security Fundamentals [View Reference]

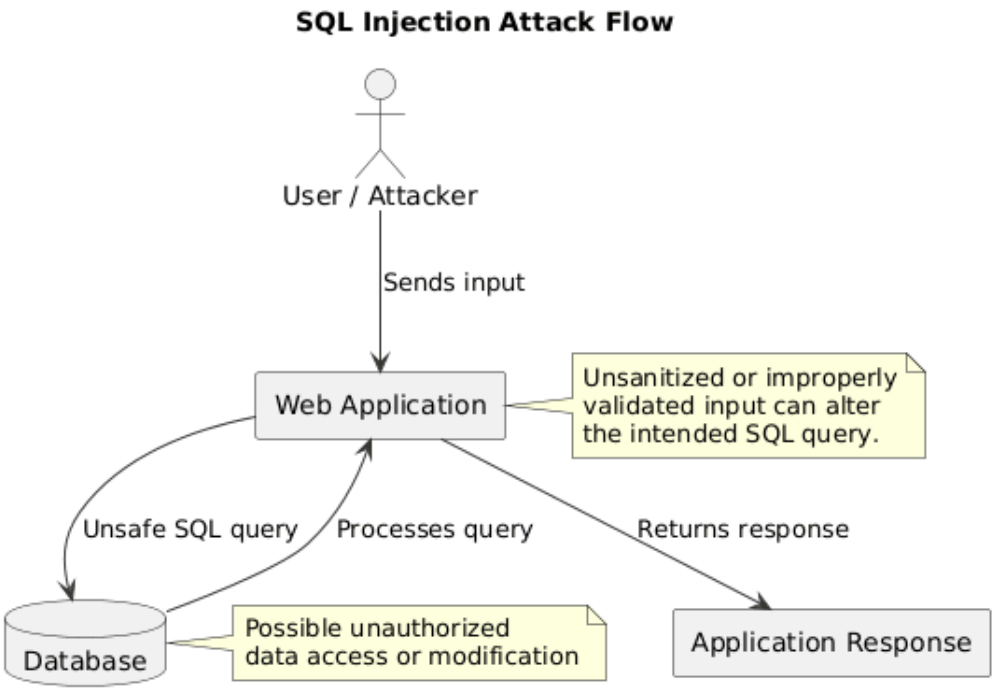


Figure 3: SQL Injection attack flow through a vulnerable web application

2.2 Cross-Site Scripting (XSS)

subsubsectionHow XSS Occurs XSS occurs when untrusted content reaches a browser-rendered context in a form that the browser interprets as executable content. The security boundary is therefore between application-controlled content and data supplied or influenced by an untrusted party.

The correct defense depends on the output context. Text inserted into ordinary HTML, an HTML attribute, JavaScript, CSS, or a URL can require different encoding approaches. This is why secure templating and context-aware output encoding are important development practices.

subsubsectionSecurity Analysis XSS can be persistent, reflected, or associated with client-side application behavior. The exact category changes how the input reaches the victim and how long the malicious content remains available. Regardless of the category, the central concern is that attacker-controlled data is interpreted in the browser as executable content.

subsubsectionDefense in Depth Output encoding should be combined with secure application design, safe templating, appropriate session protections, and browser security controls. Content Security Policy can provide an additional layer of protection, although it should complement rather than replace correct output handling.

2.2.1 Definition

Cross-Site Scripting is a web application vulnerability in which attacker-controlled content is interpreted as script by a victim's browser. The underlying problem is that untrusted data reaches a browser-rendered context without appropriate handling. The exact defense depends on where the data is inserted, such as HTML, an attribute, JavaScript, CSS, or a URL context.

2.2.2 Effect on System

The malicious script can execute within the security context of the vulnerable web application when a victim accesses affected content.

2.2.3 Possible Impact

Depending on the application and browser context, impact can include manipulation of page content, unauthorized actions performed through the victim's session, or exposure of information accessible to client-side code.

2.2.4 Basic Prevention

Use context-appropriate output encoding, validate input where appropriate, apply safe templating practices, and use suitable browser security controls such as Content Security Policy.

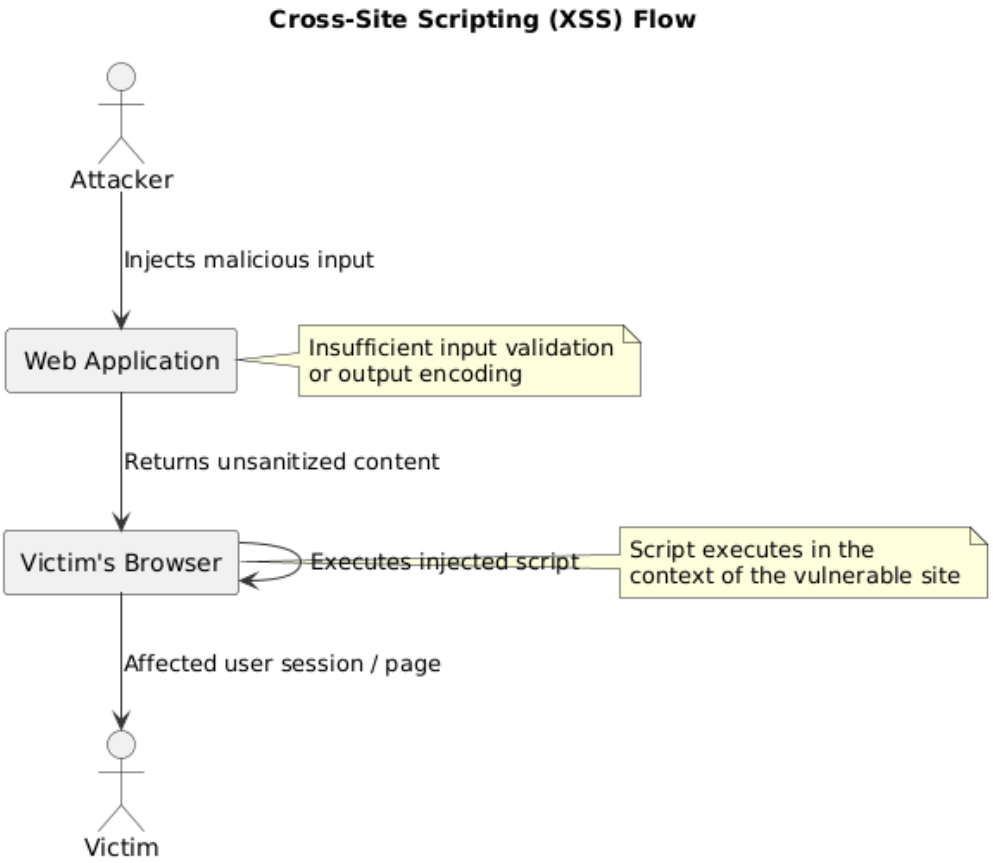


Figure 4: Cross-Site Scripting (XSS) flow through a vulnerable web application

Reference Basis: OWASP Top 10 – Injection [View Reference]

Reference / Standard: OWASP Cross Site Scripting Prevention Cheat Sheet [View Reference]

Learning Video: Professor Messer – Web Security Concepts [View Reference]

2.3 Broken Authentication

subsubsectionAuthentication and Session Management Authentication establishes a user's identity while session management maintains the authenticated state after login. A secure implementation must protect both stages. A strong password policy is not sufficient if session identifiers are predictable or remain valid for an excessive period.

subsubsectionCommon Failure Conditions Examples include weak password-recovery processes, inadequate protection against credential attacks, insecure session identifiers, missing session expiration, failure to invalidate sessions after logout, and inconsistent enforcement of authentication across protected functions.

subsubsectionSecurity Analysis Authentication weaknesses can have a direct relationship with authorization. If an attacker obtains or hijacks a valid session, the application may treat the attacker as the legitimate user. The resulting impact depends on the privileges and data available to that account.

subsubsectionDefensive Design Secure password storage, multi-factor authentication where appropriate, secure session handling, rate limiting, account protection, and careful recovery mechanisms provide multiple layers of defense. Authentication controls should also be reviewed whenever application functionality changes.

2.3.1 Definition

Broken authentication refers to weaknesses in authentication or session-management controls that can allow unauthorized users to access accounts or protected functionality. Authentication weaknesses can occur at login, password recovery, session creation, session termination, or other identity-related functions. Because these controls establish and maintain user identity, weaknesses can directly affect access to protected resources.

2.3.2 Effect on System

Weak credential handling, poor session management, predictable tokens, or inadequate authentication controls can undermine account security.

2.3.3 Possible Impact

Possible impacts include account takeover, unauthorized access to protected resources, privilege misuse, and exposure of user information.

2.3.4 Basic Prevention

Use strong authentication controls, secure password storage, multi-factor authentication where appropriate, secure session handling, appropriate session expiration, and protection against credential attacks.

Reference Basis: OWASP Top 10 – Identification and Authentication Failures [View Reference]

Reference / Standard: OWASP Authentication Cheat Sheet [View Reference]

Learning Video: Professor Messer – Authentication and Security Fundamentals [View Reference]

2.4 Security Misconfiguration

subsubsectionWhere Misconfiguration Appears Security misconfiguration can exist at almost every layer of an environment. Operating systems, web servers, databases, cloud resources, containers, network devices, and applications can all contain settings that unnecessarily increase exposure.

subsubsectionConfiguration Drift A system may begin with a secure baseline but become less secure as administrators make changes over time. Temporary troubleshooting settings, forgotten test accounts, unnecessary services, and undocumented configuration changes can create configuration drift.

subsubsectionSecurity Hardening Hardening reduces unnecessary exposure by disabling unused services, changing default credentials, limiting permissions, restricting management interfaces, removing unnecessary software, and applying a documented secure configuration baseline. Periodic review is important because secure configuration is a continuous maintenance activity rather than a one-time task.

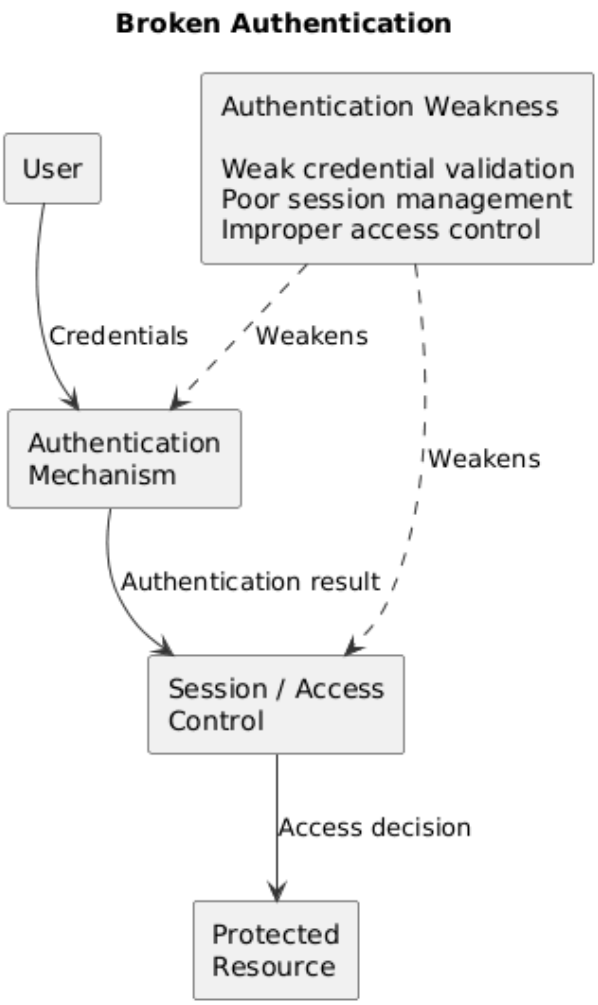


Figure 5: Broken authentication and its effect on access control

2.4.1 Definition

Security misconfiguration occurs when security settings are incorrectly implemented, left at insecure defaults, unnecessarily exposed, or inconsistently maintained. This category covers insecure settings in applications, operating systems, servers, databases, cloud resources, and network devices. It can occur when default settings remain enabled, unnecessary services are exposed, permissions are too broad, or security hardening is incomplete.

2.4.2 Effect on System

Examples include default credentials, unnecessary services, verbose error messages, exposed administrative interfaces, permissive permissions, and insecure server settings.

2.4.3 Possible Impact

Misconfiguration can increase the attack surface and may enable unauthorized access, information disclosure, or system compromise.

2.4.4 Basic Prevention

Use secure baseline configurations, remove unnecessary services, change default credentials, apply least privilege, disable unnecessary features, and review configurations regularly.

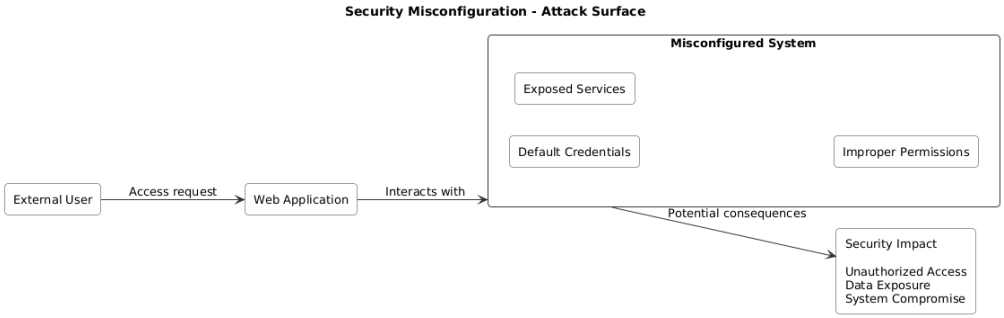


Figure 6: Security misconfiguration and its potential impact on application security

Reference Basis: OWASP Top 10 – Security Misconfiguration [View Reference]

Reference / Standard: CIS Benchmarks [View Reference]

Learning Video: Professor Messer – Security Configuration Fundamentals [View Reference]

2.5 Vulnerable and Outdated Components

subsubsectionSoftware Dependency Risk Modern applications depend on frameworks, libraries, plugins, operating-system packages, container images, and other third-party components. A weakness in one dependency can therefore become part of the application's overall attack surface.

subsubsectionAsset Inventory and Visibility Effective component management requires knowing which components are present and which versions are deployed. Without an accurate inventory, security teams may not know whether a newly disclosed vulnerability affects the environment.

subsubsectionPatch and Lifecycle Management Security updates should be evaluated and applied according to organizational risk and operational requirements. Unsupported components should be replaced where possible, and unnecessary dependencies should be removed. This reduces both the number of potential weaknesses and the effort required to monitor the environment.

2.5.1 Definition

This vulnerability occurs when applications depend on libraries, frameworks, plugins, operating-system components, or other software with known or unaddressed security weaknesses. Modern applications depend on libraries, frameworks, plugins, operating-system packages, and other third-party components. If a component contains a known weakness and is not maintained, the application may inherit the security risk even when its own code has no obvious defect.

2.5.2 Effect on System

A vulnerable dependency can expose the application even when the application's own code is reasonably secure.

2.5.3 Possible Impact

Potential impact ranges from information disclosure and denial of service to unauthorized access or remote code execution, depending on the component and vulnerability.

2.5.4 Basic Prevention

Maintain an inventory of dependencies, monitor security advisories, apply supported updates, remove unnecessary components, and use dependency-management and software composition practices.

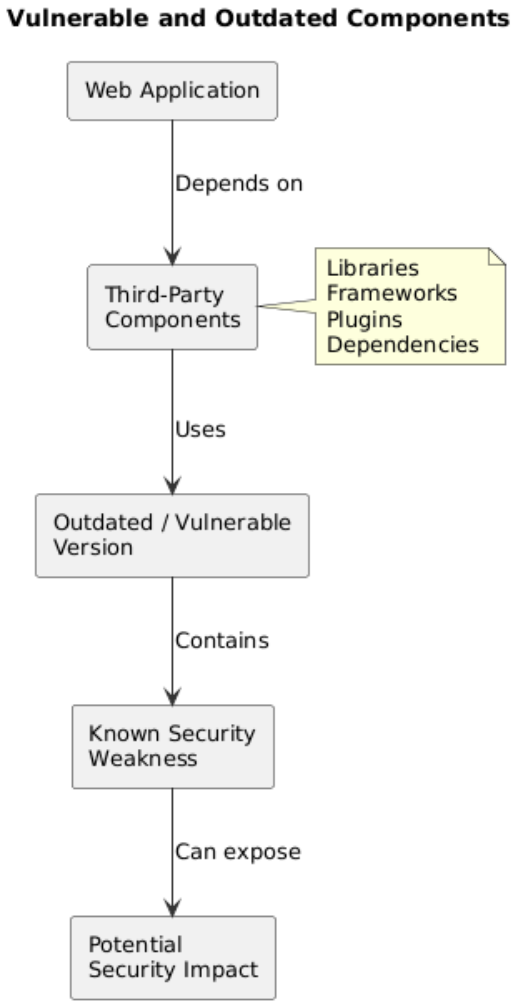


Figure 7: Vulnerable and outdated components leading to potential security impact

Reference Basis: OWASP Top 10 – Vulnerable and Outdated Components [View Reference]

Reference / Standard: CISA Known Exploited Vulnerabilities Catalog [View Reference]

Learning Video: Professor Messer – Vulnerability and Patch Management Concepts [View Reference]

2.6 Comparison of the Five Vulnerabilities

Vulnerability	Typical Area	Possible Impact	Basic Prevention
SQL Injection	Web/database	Unauthorized data access or modification	Parameterized queries; least privilege
XSS	Web application	Client-side script execution and session abuse	Output encoding; secure templating
Broken Authentication	Identity/session	Account takeover or unauthorized access	Strong authentication and session controls
Security Misconfiguration	Systems/apps	Increased attack surface and exposure	Secure baselines and configuration review
Vulnerable Components	Dependencies	Exploitation of known software weaknesses	Patch and maintain dependencies

Table 2: Comparison of common security vulnerabilities

2.7 Quick Check

1. Why is parameterized SQL preferred over string concatenation?
2. What is the role of output encoding in XSS prevention?
3. Name two controls that improve authentication security.
4. Give two examples of security misconfiguration.
5. Why should third-party components be maintained and updated?

2.8 Mini Practical – Vulnerability-to-Control Mapping

Finding Type	Security Question	Example Control
SQL Injection	Can input change database commands?	Parameterized queries
XSS	Can untrusted content execute in a browser?	Context-aware output encoding
Broken Authentication	Can an unauthorized user obtain valid access?	Strong authentication and session management
Security Misconfiguration	Are insecure settings increasing exposure?	Hardened baseline configuration
Outdated Component	Is a known vulnerable dependency deployed?	Patch and dependency management

Table 3: Mapping common vulnerabilities to primary security controls

subsectionSecure Development and Operations Perspective The five vulnerability categories demonstrate that security is shared across development and operations. Developers influence input handling, output encoding, authentication logic, and dependency selection. System administrators and security teams influence configuration, patching, access control, monitoring, and vulnerability management.

subsubsectionContinuous Improvement The presence of a vulnerability should be treated as an opportunity to improve the underlying control. If a component repeatedly becomes outdated, the organization may need better dependency inventory and patch processes. If configuration weaknesses recur, a

hardened baseline and configuration monitoring may be more effective than repeatedly correcting individual systems.

newpage
subsectionExtended Technical Analysis: Common Vulnerability Classes The five selected vulnerability classes represent different layers of a modern application environment. SQL Injection is primarily associated with the interaction between application input and databases. XSS involves the handling of untrusted content by a browser. Broken authentication affects identity and session controls. Security misconfiguration concerns insecure deployment or operational settings. Vulnerable and outdated components address weaknesses inherited from software dependencies.

subsubsectionApplication Input as a Security Boundary User-controlled input should always be treated as untrusted until it has been safely processed for its intended context. The security problem is not limited to web forms. Inputs can arrive through URLs, APIs, uploaded files, headers, mobile clients, background jobs, or integrations with other services.

subsubsectionIdentity as a Security Boundary Authentication and session management protect the boundary between an identified user and a protected application. Weaknesses in this area can turn an otherwise secure application into an environment where unauthorized users can access valid accounts.

subsubsectionConfiguration as a Security Boundary Configuration determines how software behaves in its deployed environment. A secure application can still become exposed if debugging is enabled, administrative interfaces are public, permissions are excessive, or unnecessary services remain active.

subsubsectionDependencies as an Extended Attack Surface Third-party components extend the attack surface beyond code written by the organization's developers. Dependency inventory, version tracking, security advisories, and patch management are therefore important parts of application security.

subsectionDefense-in-Depth Mapping
begintable[H]
centering
small
begintabularx
textwidth>
bfseriesp3.3cmp3.3cmX
toprule Vulnerability Primary Boundary Supporting Controls

midrule SQL Injection Application–database Parameterized queries, least privilege, secure coding review, monitoring.

XSS Application–browser Context-aware output encoding, safe templating, session protection, browser controls.

Broken Authentication User–application Strong authentication, MFA where appropriate, secure sessions, rate limiting.

Security Misconfiguration Deployment–service Secure baselines, configuration review, access restrictions, hardening.

Outdated Components Software supply chain Inventory, patch management, supported versions, dependency monitoring.

bottomrule
endtabularx
captionDefense-in-depth controls for the selected vulnerability classes
endtable

subsectionProfessional Review Approach A professional analyst should avoid reporting only the vulnerability name. The report should explain where the weakness exists, what evidence supports it, how the weakness could affect the system, and what control should reduce the exposure. This creates a clear relationship between detection and remediation.

subsectionTask 2 Learning Outcome Task 2 demonstrates that vulnerabilities can arise from application logic, identity controls, deployment configuration, and software dependencies. Understanding the layer at which a weakness occurs makes it easier to choose an appropriate control and communicate the remediation to the responsible technical team.

newpage
subsectionCase Study: Layered Application Security A modern web application can be viewed as several connected layers: user input, application logic, authentication, server configuration, database services, and third-party components. A weakness at any one layer can influence the security of the others.

For example, SQL Injection involves the application's interaction with a database. XSS involves the path from application-controlled data to a user's browser. Broken authentication affects the identity boundary. Misconfiguration affects the way the application or server is deployed. Vulnerable components extend the attack surface through software that the application depends on.

This layered view is useful because remediation should address the underlying control. If an application is vulnerable to SQL Injection, simply blocking one suspicious string is not a complete solution; the database interaction should be designed so that data cannot unexpectedly become query syntax. Similarly, if a system is repeatedly affected by outdated components, the organization may need stronger dependency inventory and update processes.

subsectionSecurity Review Matrix
begintable[H]
centering
small
begintabularx
textwidth>
bfseriesp3.1cmp3.0cmX
toprule Security Layer Example Weakness Review Focus

midrule Input Handling SQL Injection, unsafe input processing Data validation and separation of data from commands.

Browser Output XSS Context-aware output encoding and safe rendering.

Identity Broken authentication Credential, session and account controls.

Deployment Security misconfiguration Hardened baseline and configuration consistency.

Dependencies Outdated components Version inventory, advisories and patch management.

bottomrule
endtabularx
captionLayered review model for common application vulnerabilities
endtable

The case study reinforces the importance of defense in depth. Security is stronger when multiple independent controls reduce the probability and impact of failure.

2.9 Task 2 Summary

SQL Injection, XSS, broken authentication, security misconfiguration, and vulnerable components represent different classes of weaknesses. Preventive controls should address both technical implementation and ongoing security maintenance.

3 Task 3: Vulnerability Assessment Process

3.1 Introduction

A vulnerability assessment follows a structured lifecycle so that findings are not only discovered but also analyzed, prioritized, remediated, and verified. The process should operate within an explicitly authorized scope.

3.2 Step 1 – Identify

subsubsectionAsset Discovery in Practice Identification begins with understanding what exists inside the authorized scope. This can include hostnames, IP addresses, applications, operating systems, network services, databases, cloud resources, and other technology assets. The assessor should also understand ownership and the business purpose of important systems.

subsubsectionScope Validation Before scanning, the target list should be compared with the written authorization. This reduces the risk of accidentally assessing an unrelated system and establishes a clear boundary for later evidence and reporting.

subsubsectionExpected Documentation A useful identification stage produces an asset and scope record that can be referenced throughout the assessment. This record becomes the foundation for scan configuration, finding ownership, prioritization, and remediation tracking.

3.2.1 What Happens

The assessor identifies assets, systems, applications, network ranges, services, and assessment scope.

3.2.2 Why It Is Important

An accurate asset inventory prevents important systems from being overlooked and ensures that testing stays within authorization boundaries.

3.2.3 Expected Outcome

A defined list of targets and an agreed assessment scope.

3.3 Step 2 – Scan

subsubsectionScan Selection The scan configuration should match the assessment objective. A broad network scan may identify services and basic weaknesses, while a more detailed vulnerability assessment can provide additional information about software, configurations, and known vulnerabilities.

subsubsectionOperational Considerations Scanning generates traffic and can affect fragile systems if configured aggressively. The assessor should consider scan timing, system sensitivity, network capacity, and the rules of engagement before starting the assessment.

subsubsectionExpected Evidence A completed scan should provide a target record, scan status, findings, severity information, and supporting evidence. These results should then be reviewed rather than accepted blindly.

3.3.1 What Happens

Authorized scanning tools inspect the target for known vulnerabilities, exposed services, software versions, configuration weaknesses, and other indicators.

3.3.2 Why It Is Important

Automated scanning provides broad coverage and helps identify weaknesses that may require further validation.

3.3.3 Expected Outcome

A collection of potential findings requiring analysis.

3.4 Step 3 – Analyze

subsubsectionValidation of Findings Analysis determines whether a reported finding is relevant and supported by evidence. The assessor reviews the affected asset, service, software version, plugin information, configuration, and available supporting output.

subsubsectionFalse Positives and Context Automated tools can report findings that do not apply to the target or whose practical risk is lower than the initial scanner classification suggests. Manual validation and contextual review help reduce false positives and improve report quality.

subsubsectionExpected Evidence The analysis stage should result in findings that are sufficiently understood to support prioritization and remediation. Important findings should contain enough evidence that another reviewer can understand why the issue was reported.

3.4.1 What Happens

Findings are reviewed for accuracy, context, affected assets, severity, and evidence.

3.4.2 Why It Is Important

Automated scanners can produce false positives or findings whose importance depends on the environment.

3.4.3 Expected Outcome

Validated and contextualized findings.

3.5 Step 4 – Prioritize

subsubsectionRisk-Based Prioritization Prioritization should consider more than a severity label. Technical severity, exploitability, exposure, asset criticality, likelihood, potential impact, and existing controls can all influence the order in which findings are addressed.

subsubsectionRemediation Planning A practical remediation list should distinguish urgent findings from issues that can be handled through normal maintenance. This allows security teams to use limited resources efficiently while keeping lower-priority findings visible and tracked.

subsubsectionExpected Outcome The expected result is a risk-ranked set of findings with clear ownership and an appropriate remediation path.

3.5.1 What Happens

Findings are ranked using severity, likelihood, exploitability, exposure, asset importance, and potential impact.

3.5.2 Why It Is Important

Prioritization allows security teams to address the most important risks first.

3.5.3 Expected Outcome

A risk-ranked remediation list.

3.6 Step 5 – Remediate

subsubsectionTypes of Remediation Remediation may involve installing a security update, upgrading software, changing configuration, restricting access, removing an unnecessary service, strengthening authentication, or applying another compensating control. The appropriate action depends on the root cause of the finding.

subsubsectionChange Management Remediation should be performed carefully in operational environments. Changes may need testing, approval, scheduling, and rollback planning. Security improvement should not introduce an avoidable availability problem.

subsubsectionExpected Outcome The weakness should be removed or its exposure reduced to a level that the organization considers acceptable. The change should also be documented so that it can be verified later.

3.6.1 What Happens

The organization applies patches, configuration changes, access controls, compensating controls, or other corrective measures.

3.6.2 Why It Is Important

Identification alone does not improve security unless weaknesses are reduced or eliminated.

3.6.3 Expected Outcome

The identified weakness is corrected or its risk is reduced to an acceptable level.

3.7 Step 6 – Re-scan and Verify

subsubsectionVerification Method Re-scanning provides evidence that the corrective action changed the security condition that originally produced the finding. Verification should use an assessment method appropriate to the original weakness.

subsubsectionWhy a Finding May Remain A finding may remain because a patch was not applied correctly, a service was not restarted, a vulnerable component still exists, or the underlying configuration was not fully changed. Verification helps identify these situations.

subsubsectionExpected Outcome The final result should clearly indicate whether the weakness was remediated, reduced, accepted, or still requires further action.

3.7.1 What Happens

The affected system is assessed again to determine whether the vulnerability has been remediated.

3.7.2 Why It Is Important

Verification provides evidence that remediation was effective.

3.7.3 Expected Outcome

A verified remediation status and an updated security assessment.

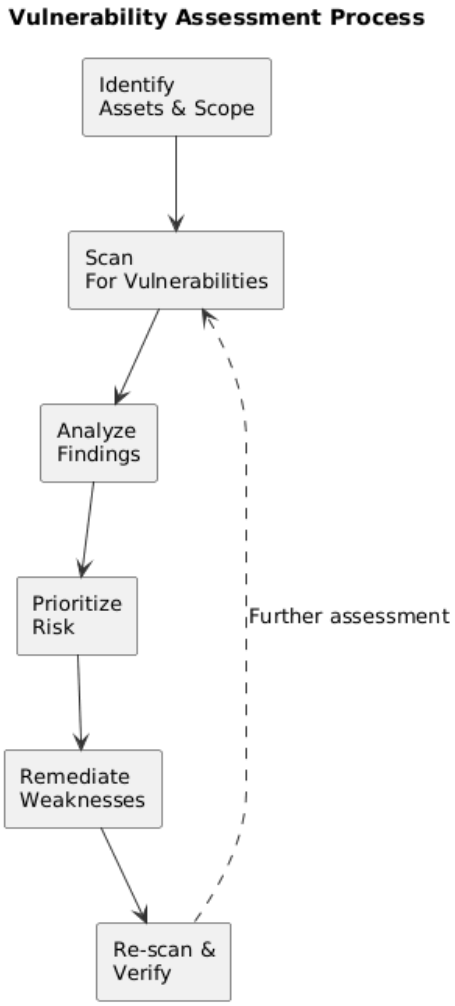


Figure 8: Vulnerability assessment lifecycle from identification to verification

3.8 Quick Check

1. Why is asset identification important?
2. What is the purpose of vulnerability scanning?
3. Why should findings be analyzed before remediation?
4. What factors can influence prioritization?
5. Why is re-scanning necessary?

subsectionContinuous Vulnerability Management The six-step lifecycle is not necessarily a process that ends after one re-scan. New software versions, new vulnerabilities, configuration changes, and new assets continuously change the security condition of an environment. Mature vulnerability management therefore repeats the cycle and tracks findings over time.

subsubsectionMetrics and Tracking Useful operational measures can include the number of open Critical and High findings, average remediation time, percentage of assets assessed, number of overdue findings, and percentage of findings verified after remediation. Such measurements help security teams demonstrate whether vulnerability management is improving.

newpage
subsectionExtended Technical Analysis: Vulnerability Assessment Lifecycle The vulnerability assessment lifecycle is designed to convert a large and potentially noisy set of technical observations into a controlled remediation process. Each stage has a distinct purpose, and skipping a stage can reduce the quality of the final result.

subsubsectionWhy Identification Comes First A scanner cannot assess what it does not know about. Asset identification establishes the boundary of the assessment and creates an inventory against which coverage can be measured. It also helps prevent unauthorized testing because the target list can be checked before scanning begins.

subsubsectionWhy Scanning Is Not the Final Answer Automated scanning is efficient because it can examine many systems and recognize patterns associated with known weaknesses. However, automated results can contain false positives, incomplete context, or findings whose practical importance differs between environments. Human analysis remains necessary.

subsubsectionWhy Prioritization Is Necessary Large environments can produce many findings. Treating every result as equally urgent would make remediation difficult and could distract teams from the weaknesses that present the greatest exposure. Risk-based prioritization creates a practical order for action.

subsubsectionWhy Remediation Needs Verification A remediation ticket being marked “complete” is not technical evidence that the weakness has disappeared. Re-scanning or another suitable verification method provides evidence that the security condition has changed.

subsectionAssessment Quality Factors
begintable[H]
centering
small
begintabularx
textwidth>
bfseriesp3.3cmX X
toprule Quality Factor Question Why It Matters

midrule Scope Accuracy Were only authorized systems assessed? Prevents accidental out-of-scope

activity.

Coverage Were important assets included? Reduces blind spots in the assessment.

Finding Validity Does the evidence support the finding? Reduces false positives and improves trust.

Prioritization Was context considered? Focuses resources on meaningful risk.

Remediation Is the recommendation practical? Converts findings into corrective action.

Verification Was remediation re-tested? Provides evidence that the issue was addressed.

bottomrule
endtabularx
captionFactors that influence vulnerability assessment quality
endtable

subsectionOperational Tracking Organizations can maintain vulnerability records over time. A finding may move through states such as Open, Assigned, In Progress, Remediated, and Verified. Tracking these states creates accountability and allows security teams to measure remediation performance.

subsectionTask 3 Learning Outcome Task 3 establishes vulnerability assessment as a continuous process rather than a single scan. The assessor identifies the scope, scans, analyzes evidence, prioritizes findings, supports remediation, and verifies the result. This lifecycle is directly reflected in the Nessus practical performed in Task 5.

newpage
subsectionCase Study: Managing a Vulnerability Assessment Finding Suppose a vulnerability scanner identifies a high-severity weakness on a server. The finding should first be linked to the correct asset. The assessor then reviews the evidence to confirm that the vulnerable component or configuration is actually present. If the finding is valid, the assessor considers exposure, asset importance, exploitability, and impact before assigning a remediation priority.

The remediation team may apply a software update, change a configuration, restrict network access, or implement another corrective measure. After the change, the assessor performs a re-scan or another suitable verification activity. If the finding disappears and the evidence supports the change, the remediation can be marked as verified.

This case illustrates why each stage of the lifecycle is important. Identification provides scope, scanning provides observations, analysis provides confidence, prioritization provides order, remediation reduces exposure, and verification provides evidence of improvement.

subsectionLifecycle Review Questions
beginenumerate
item What happens if the asset inventory is incomplete?
item What happens if a scan finding is accepted without validation?
item What happens if all findings are treated as equally urgent?
item What happens if remediation is performed without verification?
item How can repeated findings indicate a process weakness rather than only a technical weakness?
endenumerate

The answers reinforce that vulnerability management is both a technical and operational discipline. Repeated weaknesses can indicate missing patch processes, unclear ownership, weak configuration

management, or inadequate security awareness.

3.9 Task 3 Summary

The vulnerability assessment process follows Identify → Scan → Analyze → Prioritize → Remediate → Re-scan and Verify. This lifecycle turns scanner output into an actionable and verifiable security-improvement process.

4 Task 4: Introduction to Penetration Testing

4.1 Scope, Rules of Engagement and Authorization

A penetration test should begin with a clear agreement between the tester and the system owner.

4.1.1 Scope

Scope identifies the systems, applications, addresses, accounts, environments, and testing windows that are included.

4.1.2 Out-of-Scope Systems

Explicit exclusions prevent accidental testing of systems that the tester is not authorized to assess.

4.1.3 Testing Constraints

The engagement may restrict techniques that could cause service interruption, data modification, destructive changes, or uncontrolled propagation.

4.2 Why Written Permission Is Important

Security testing creates traffic and may involve techniques that resemble malicious activity. Written authorization provides evidence that the activity is approved and establishes the boundaries of the assessment.

4.2.1 Authorization Details

A written authorization should identify the responsible organization or owner, the approved targets, assessment dates, tester or team, and relevant conditions.

4.2.2 Emergency Contact

The tester should know whom to contact if the assessment causes an unexpected service issue or discovers a severe security condition requiring immediate attention.

4.3 VA and PT – Practical Difference

A vulnerability assessment can identify many potential weaknesses across a broad environment. A penetration test generally selects relevant weaknesses or attack paths and validates their practical security significance under controlled conditions.

4.3.1 Coverage

Vulnerability assessment generally emphasizes breadth and identification. Penetration testing emphasizes validation, attack-path reasoning, and impact.

4.3.2 Output

A vulnerability assessment commonly produces prioritized findings and remediation recommendations. A penetration test may additionally document validated attack paths, evidence of exploitation, security-control observations, and business impact.

4.4 Ethical and Professional Considerations

A security tester must protect client information, minimize unnecessary impact, stay within scope, document actions accurately, and report findings responsibly. Professional testing is intended to improve security rather than create uncontrolled damage.

4.5 What is Penetration Testing?

subsubsectionProfessional Definition Penetration testing is a controlled and authorized security activity in which a tester evaluates whether weaknesses can be practically abused and what security consequences may follow. The tester works within a defined scope and uses evidence to demonstrate security conditions rather than attempting uncontrolled compromise.

subsubsectionWhat Makes It Different from Scanning A scanner can identify indicators of vulnerability at scale, but penetration testing requires human reasoning to select relevant attack paths and interpret how weaknesses interact. The tester considers the target architecture, trust relationships, access controls, and potential impact.

subsubsectionExpected Value The value of penetration testing is not measured only by the number of vulnerabilities discovered. It also comes from demonstrating realistic attack paths, evaluating security controls, identifying gaps in detection or response, and giving system owners actionable remediation guidance.

4.5.1 Definition

Penetration testing is an authorized security assessment in which a tester evaluates whether security weaknesses can be practically abused within a defined scope. It is more than simply listing vulnerabilities: it focuses on validating security controls and understanding potential impact.

4.5.2 Why Penetration Testing Is Performed

Penetration testing can help organizations validate defenses, identify exploitable weaknesses, assess attack paths, test detection and response capabilities, and produce evidence-based remediation recommendations.

4.6 Vulnerability Assessment vs. Penetration Testing

subsubsectionMethodology Comparison Vulnerability assessment generally aims to identify and prioritize a broad set of weaknesses. Penetration testing generally focuses on validating selected weaknesses and understanding whether they can be combined into a meaningful attack path.

subsubsectionCoverage and Depth A vulnerability assessment is often broader and more repeatable, making it useful for continuous vulnerability management. Penetration testing is typically more targeted and investigative, with greater emphasis on controlled validation and impact.

subsubsectionComplementary Use The two approaches are complementary rather than competing activities. Vulnerability assessments provide broad visibility, while penetration tests provide deeper validation of selected security conditions.

Vulnerability assessment primarily focuses on discovering and prioritizing weaknesses. Penetration testing goes further by validating selected weaknesses through controlled testing within the approved scope.

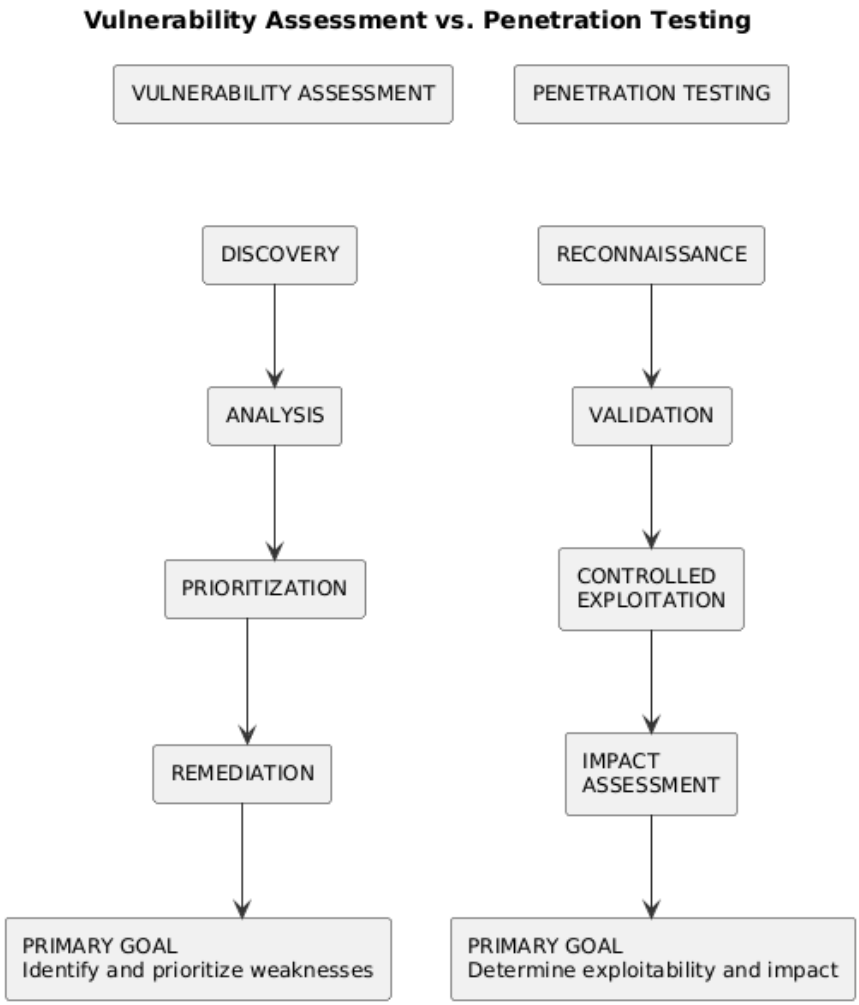


Figure 9: Comparison of vulnerability assessment and penetration testing

4.7 Authorized Security Testing

4.7.1 Written Permission

Written permission establishes that the tester is authorized to assess the specified systems. It also helps define scope, time windows, permitted techniques, contacts, and handling requirements.

4.7.2 Why Authorization Matters

Scanning, testing, or exploiting systems without permission can disrupt services, expose data, violate policy, or create legal consequences. Professional security testing therefore begins with authorization and a clearly defined scope.

4.8 Penetration Testing Lifecycle

subsubsectionPlanning Planning establishes objectives, scope, authorization, constraints, contacts, timing, evidence requirements, and reporting expectations. A well-defined plan reduces ambiguity and helps prevent testing outside the approved boundary.

subsubsectionReconnaissance Reconnaissance collects permitted information about the target so the tester can understand the available attack surface. In a professional engagement, the tester limits collection to the information allowed by the rules of engagement.

subsubsectionScanning Scanning identifies reachable services, technologies, versions, and potential weaknesses. Results from this stage guide later analysis rather than automatically proving exploitability.

subsubsectionVulnerability Analysis The tester correlates observed services and weaknesses with known security conditions and evaluates which findings are relevant enough for controlled validation.

subsubsectionSecurity Testing Testing validates selected weaknesses using controlled methods. The tester should use the least intrusive approach that can demonstrate the security issue and should stop or escalate when an action could create unexpected impact.

subsubsectionReporting Reporting converts technical observations into findings that explain evidence, risk, potential impact, and remediation. A clear report allows technical teams to reproduce the context and management to understand the priority.

4.8.1 Planning

Define objectives, scope, authorization, constraints, contacts, and reporting requirements.

4.8.2 Reconnaissance

Collect permitted information about the target environment to understand the attack surface.

4.8.3 Scanning

Use appropriate authorized techniques to identify services, technologies, and potential weaknesses.

4.8.4 Vulnerability Analysis

Analyze discovered information and determine which weaknesses warrant controlled validation.

4.8.5 Security Testing

Perform controlled testing against approved targets while minimizing disruption and avoiding unnecessary impact.

4.8.6 Reporting

Document findings, evidence, risk, business/technical impact, and remediation recommendations.

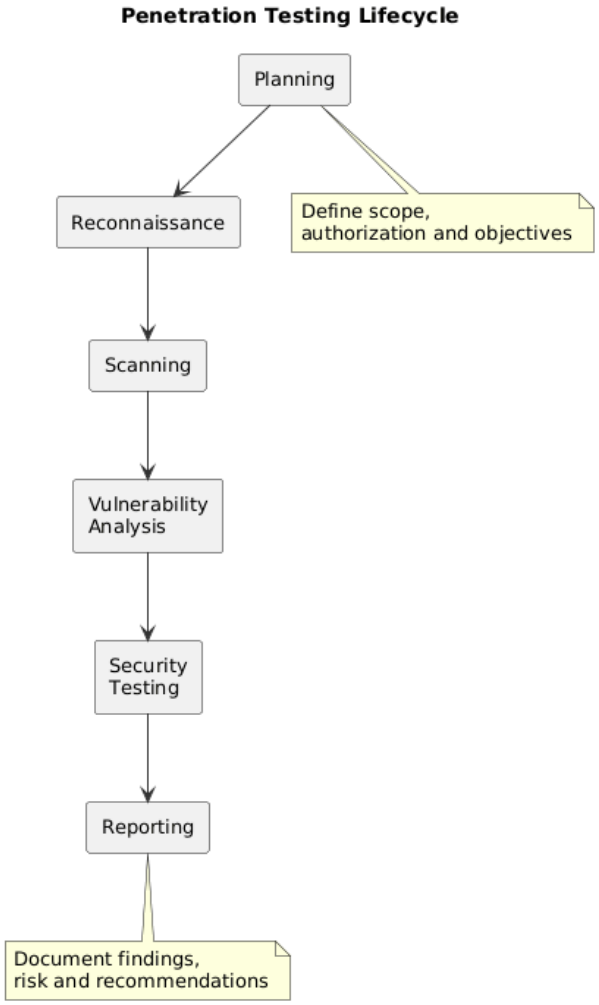


Figure 10: Penetration testing lifecycle from planning to security reporting

4.9 Authorized Penetration Testing Environment

A professional assessment should use an explicitly authorized target such as an organization's approved system, the tester's own computer, an intentionally vulnerable laboratory, or an approved training platform.

Reference Basis: NIST SP 800-115 – Technical Guide to Information Security Testing and Assessment [View Reference]

Reference / Standard: NIST SP 800-115 [View Reference]

Learning Video: Professor Messer – Penetration Testing and Security Assessment Concepts [View Reference]

4.10 Quick Check

1. What is the main difference between vulnerability assessment and penetration testing?
2. Why is written permission important?
3. What is defined during the planning phase?

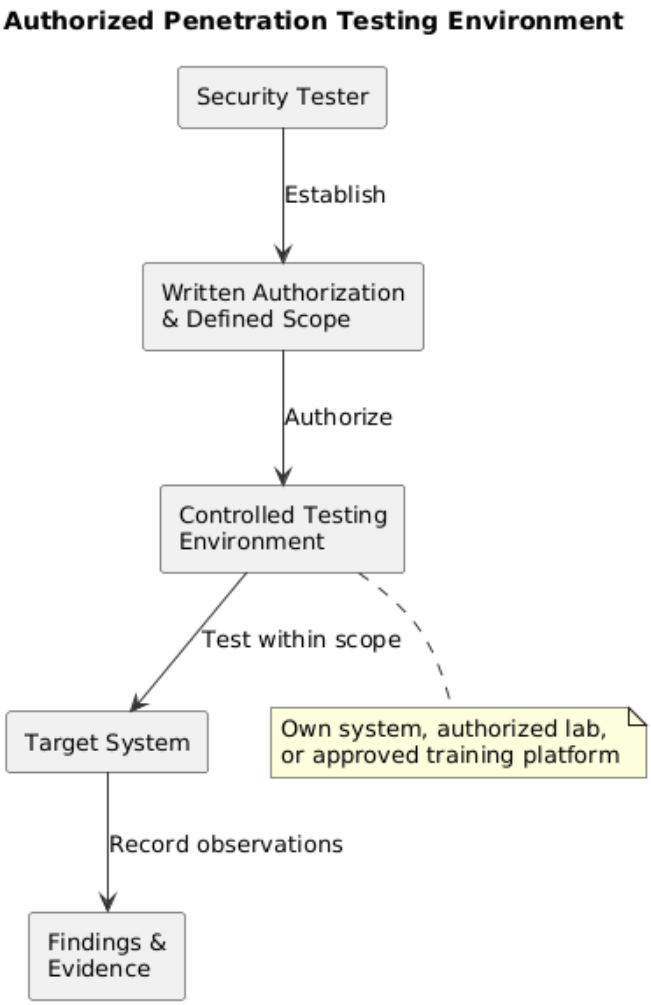


Figure 11: Authorized penetration testing workflow within a defined security scope

4. Why should penetration testing remain within a defined scope?
5. What should be included in the final report?

subsectionProfessional Penetration Testing Mindset A professional penetration tester combines technical knowledge with careful decision-making. The objective is not to "break everything" but to determine what can be demonstrated safely within the agreed scope and to provide evidence that helps the organization improve its defenses.

subsubsectionCommunication During Testing Clear communication is important when a test produces an unexpected condition. Testers should know escalation contacts and should document significant events. This reduces the chance that an assessment itself becomes an operational incident.

newpage
subsectionExtended Technical Analysis: Penetration Testing Methodology Penetration testing combines technical assessment with controlled security validation. The tester must understand the target, respect authorization boundaries, select appropriate techniques, record evidence, and communicate findings accurately. The methodology therefore contains both technical and professional controls.

subsubsectionPlanning as a Security Control Planning is not administrative overhead. It is itself a security control because it establishes what the tester is permitted to do. Clear scope and constraints reduce the possibility of accidental disruption and make the final assessment defensible.

subsubsectionReconnaissance and Attack Surface Understanding Reconnaissance helps the tester understand what is exposed and how components relate to each other. In an authorized assessment, reconnaissance should remain within the approved information-gathering boundaries. The goal is to understand the target rather than collect unnecessary information.

subsubsectionScanning and Enumeration Scanning identifies technical characteristics such as open services, versions, and potential weaknesses. Enumeration adds detail about available resources and service behavior. These stages support vulnerability analysis but do not automatically establish exploitability.

subsubsectionControlled Validation When a weakness is selected for validation, the tester should use the least intrusive method capable of demonstrating the security condition. If the test could alter data, interrupt availability, or expose sensitive information, the tester should follow the agreed rules of engagement and escalation procedure.

subsubsectionReporting as the Final Deliverable A penetration test produces value only when its findings can be understood and acted upon. Reports should separate evidence from interpretation, explain impact, identify affected assets, and recommend practical corrective actions.

subsectionVA and PT Decision Table
begintable[H]
centering
small
begintabularx
textwidth>
bfseriesp3.0cmX X
toprule Characteristic Vulnerability Assessment Penetration Testing

midrule Primary Goal Identify and prioritize weaknesses. Validate selected weaknesses and attack paths.

Typical Coverage Broad and repeatable. More targeted and investigative.

Automation Often heavily automated. Uses automation plus human analysis.

Exploitation Not required for every finding. Controlled validation may be performed when authorized.

Main Output Vulnerability findings and remediation priorities. Validated findings, evidence, impact and recommendations.

bottomrule
endtabularx
captionPractical comparison of vulnerability assessment and penetration testing
endtable

subsectionTask 4 Learning Outcome Task 4 establishes that penetration testing is not simply the use of offensive tools. It is a controlled professional activity governed by scope, authorization, methodology, evidence handling, safety, and reporting. These principles are essential for responsible cybersecurity practice.

newpage
subsectionCase Study: Authorized Penetration Test Planning Imagine that an organization authorizes a security team to test a web application during a defined maintenance window. Before any technical activity begins, the team documents the application, approved addresses, test dates, allowed techniques, excluded systems, emergency contacts, and evidence-handling requirements.

During reconnaissance, the team identifies the approved application's exposed functionality. Scanning then provides information about services and possible weaknesses. The tester analyzes those results and selects a small number of relevant weaknesses for controlled validation. If a test could affect availability or modify data, the tester follows the agreed restrictions and escalation process.

At the end of the engagement, the team reports what was observed, what was successfully validated, what impact could result, and what remediation is recommended. This example demonstrates that authorization and methodology are not separate from technical skill; they are part of professional security testing itself.

subsectionEthical Decision Framework When considering a testing action, a professional tester should ask:
beginenumerate
item Is this action explicitly within the authorized scope?
item Is the target definitely the approved system?
item Is the action necessary to demonstrate the finding?
item Could it modify data or affect availability?
item Is there a safer alternative that provides sufficient evidence?
item Who should be contacted if unexpected impact occurs?
endenumerate

This decision framework helps maintain a clear boundary between authorized security testing and uncontrolled activity.

4.11 Task 4 Summary

Penetration testing is an authorized, controlled security assessment. A professional lifecycle begins with planning and authorization, progresses through reconnaissance, scanning, vulnerability analysis and controlled testing, and concludes with evidence-based reporting.

5 Task 5: Safe Practical Activity

5.1 Introduction

For this practical activity, Nessus Essentials was used as the primary vulnerability assessment tool in a controlled laboratory environment. The target was an intentionally vulnerable Metasploitable system. The activity was performed for security learning and assessment purposes within the lab environment.

5.2 Tool Used – Nessus Essentials

subsectionNessus Interface and Assessment Components The Nessus interface used in the practical provided several stages of the assessment workflow. The setup interface established the Nessus deployment, the Events page showed component and plugin activity, the scan page displayed the configured assessment, and the completed results page presented the findings.

subsubsectionScan Configuration The practical used a Basic Network Scan against the authorized Metasploitable laboratory target. The scan was configured and executed within the local lab environment rather than against a public or third-party system.

subsubsectionResult Interpretation Nessus organizes findings by severity and provides plugin-level information for individual results. This makes it possible to move from a high-level overview to detailed evidence without treating the overall vulnerability count as the complete security conclusion.

subsubsectionWhy Plugin Evidence Matters Plugin details provide the technical context needed for reporting. Information such as plugin ID, description, solution, affected host, output, and risk metrics can be recorded as evidence for a finding.

Nessus is a vulnerability assessment platform used to identify security weaknesses in systems and applications. In this practical, Nessus was configured as a local scanner and used to perform a Basic Network Scan against the laboratory target.

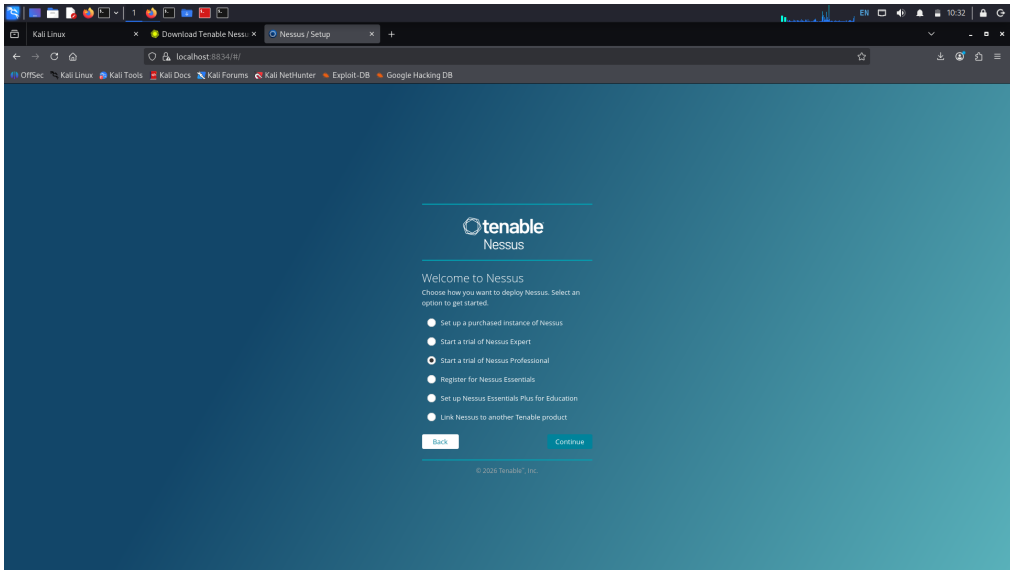


Figure 12: Nessus setup screen used to configure the Nessus deployment

5.3 Practical Objective

The objective of this activity was to perform a controlled vulnerability assessment against an intentionally vulnerable Metasploitable laboratory target using Nessus Essentials. The exercise focused on identifying vulnerabilities, reviewing scanner evidence, understanding severity information, and documenting remediation recommendations.

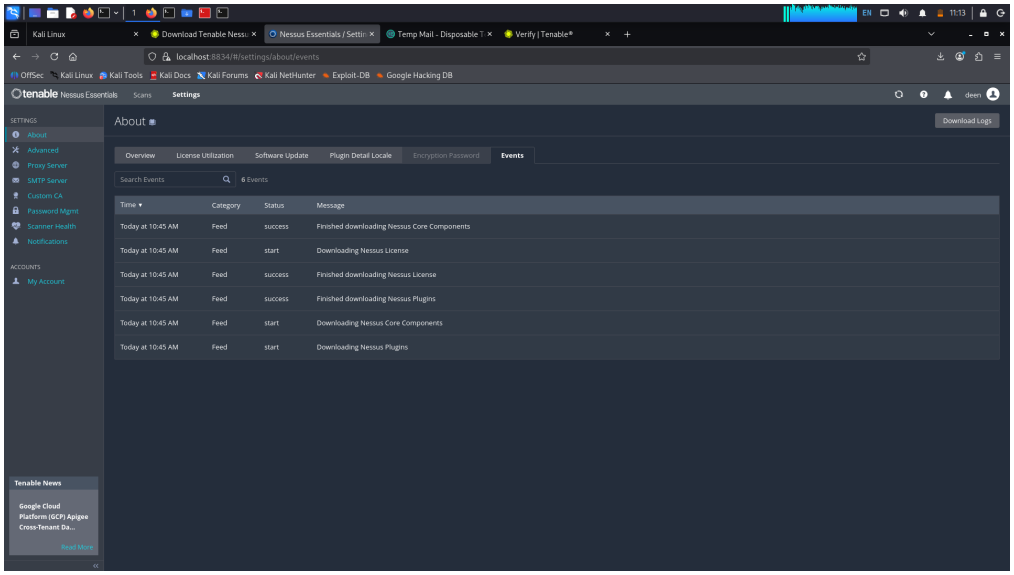


Figure 13: Nessus Events page showing successful component and plugin downloads

5.4 Why Nessus Was Selected

Nessus is appropriate for this practical because it is specifically designed for vulnerability assessment. It provides structured findings, severity information, plugin identifiers, evidence, risk metrics, and remediation guidance. This makes it suitable for demonstrating the complete assessment workflow required by the task.

5.4.1 Assessment Workflow

The practical followed the sequence:

1. Prepare and verify the Nessus installation.
2. Confirm the authorized laboratory target.
3. Create the vulnerability assessment scan.
4. Execute the scan using the selected policy.
5. Review the completed findings.
6. Examine representative Critical findings.
7. Record evidence, risk and remediation.

5.5 Nessus Assessment Observations

The Nessus interface provided a completed scan report with the number of vulnerabilities and their severity distribution. Individual plugin pages supplied additional information such as description, solution, plugin ID, CVSS, VPR, EPSS, affected host and observed output.

5.5.1 Severity Distribution

The results interface displayed Critical, High, Medium, Low, and Informational categories. These categories provide a convenient first-level method of organizing findings for review.

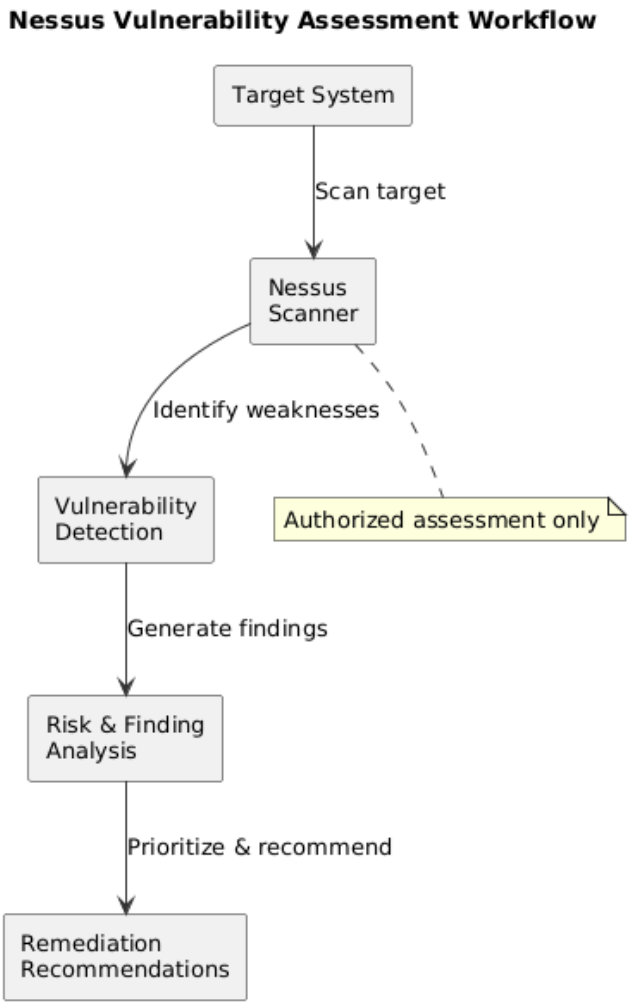


Figure 14: Nessus-based vulnerability assessment workflow

5.5.2 Plugin-Level Evidence

The plugin detail view provides information that is useful for a security finding report. This includes the plugin identifier, severity, description, solution, risk information, and output observed during the scan.

5.6 Observed Vulnerability Results

subsectionInterpreting the 72 Findings The completed Nessus scan reported 72 vulnerabilities. This number represents the findings identified by the selected scan configuration against the laboratory target at the time of assessment. It should not be interpreted as 72 separate confirmed compromises.

subsubsectionSeverity Categories The results page grouped findings into Critical, High, Medium, Low, and Informational categories. Severity helps organize the review process, but final remediation priority should also consider asset importance, exposure, exploitability, and available controls.

subsubsectionRepresentative Findings Two Critical findings were selected for detailed documentation because their plugin pages contained clear evidence, risk information, and remediation guidance. Selecting representative findings makes the report practical while avoiding unnecessary reproduction of every scanner entry.

The completed scan reported **72 vulnerabilities**. The Nessus results interface classified findings by severity and displayed additional metrics such as CVSS, VPR and EPSS where available.

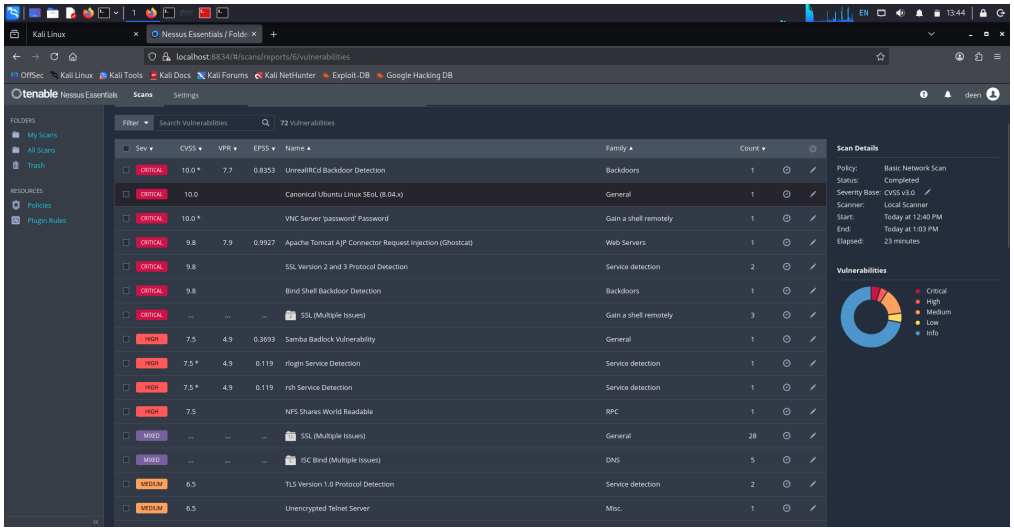


Figure 15: Completed Nessus vulnerability assessment showing the observed findings and severity distribution

5.7 Finding 1 – UnrealIRCd Backdoor Detection

subsubsectionAssessment Interpretation The finding demonstrates how a vulnerability scanner can identify a security condition associated with a specific service and provide evidence at plugin level. The observed result identifies the IRC service on TCP port 6667 and associates the service with the UnrealIRCd backdoor detection plugin.

subsubsectionSecurity Significance Because the observed severity was Critical and the displayed CVSS base score was 10.0, the finding deserves immediate attention in a real environment. The laboratory context is intentionally vulnerable, so the result is useful as an example of how a high-impact finding should be documented and prioritized.

5.7.1 Finding

Nessus identified **UnrealIRCd Backdoor Detection** as a Critical finding, Plugin ID **46882**.

5.7.2 Risk Level

Critical. The Nessus result displayed a CVSS v3.0 base score of **10.0** and a VPR value of **7.7** for this finding.

5.7.3 Potential Impact

The plugin description states that the affected remote IRC service contains a backdoor that could allow arbitrary code execution on the affected host. In the observed scan output, the service was identified on port **6667/tcp**.

5.7.4 Recommendation

The Nessus solution recommends re-downloading the affected software, verifying its integrity using published checksums, and reinstalling it. In a real environment, affected software should be removed or upgraded according to the vendor's supported remediation guidance.

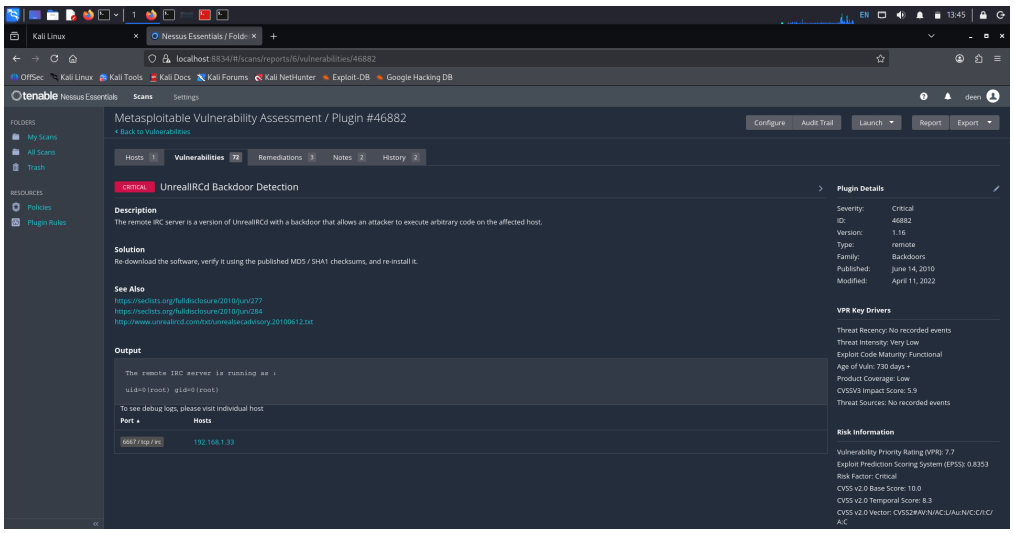


Figure 16: Nessus detail view for the Critical UnrealIRCd Backdoor Detection finding

5.8 Finding 2 – Apache Tomcat AJP Connector Request Injection

subsubsectionAssessment Interpretation The Ghostcat finding demonstrates the importance of reviewing both the affected service and the configuration surrounding it. The Nessus plugin identifies a vulnerable AJP connector and provides information about the possible consequences and recommended remediation.

subsubsectionSecurity Significance The observed CVSS score of 9.8 and the displayed VPR and EPSS values provide additional context for prioritization. In an operational environment, the final decision would still consider whether the affected connector is reachable, what applications are hosted on the server, and what compensating controls exist.

5.8.1 Finding

Nessus identified **Apache Tomcat AJP Connector Request Injection (Ghostcat)** as a Critical finding, Plugin ID **134862**.

5.8.2 Risk Level

Critical. The displayed CVSS v3.0 base score was **9.8**, with a VPR of **7.9** and EPSS of **0.9927** in the observed result.

5.8.3 Potential Impact

The finding description indicates that a remote unauthenticated attacker could potentially read web application files from a vulnerable AJP connector. In conditions where file upload is possible, the weakness can have more serious consequences.

5.8.4 Recommendation

The Nessus solution recommends configuring the AJP connector to require authorization and/or upgrading the affected Tomcat version to a supported version that addresses the vulnerability.

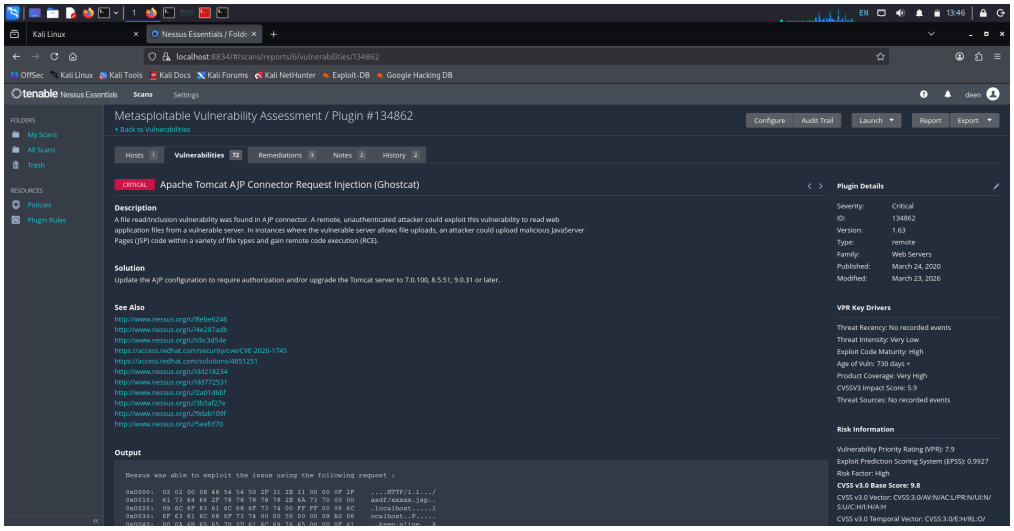


Figure 17: Nessus detail view for the Critical Apache Tomcat AJP Connector Request Injection finding

5.9 Security and Safety Rule

subsectionResponsible Use of Security Tools Security tools are dual-use technologies. The same scanner or testing framework that helps a security professional identify weaknesses can cause operational or legal problems when used against systems without authorization. The practical therefore treats authorization as a mandatory technical and professional requirement.

subsubsectionLaboratory Boundary The Metasploitable system used in this activity is intentionally vulnerable and suitable for controlled learning. The assessment remains limited to the laboratory target and does not imply permission to scan systems outside the defined environment.

subsubsectionEvidence and Privacy Assessment evidence should be stored securely and should not expose unnecessary credentials, personal information, tokens, private keys, or other sensitive material. Screenshots included in a report should be reviewed before sharing.

Scanning, testing, exploiting, or attacking systems without permission is not part of this practical activity. The exercise must remain restricted to the authorized laboratory environment.

Reference Basis: Tenable Nessus – Vulnerability Assessment Platform [View Reference]

Reference / Standard: Tenable Nessus Essentials [View Reference]

Reference / Standard: OWASP Vulnerable Web Applications Directory – Training Environments [View Reference]

Learning Video: Tenable – Nessus Vulnerability Management and Assessment Resources [View Reference]

5.10 Quick Check

1. What was the purpose of using Nessus in this practical?
2. What target was assessed?
3. How many vulnerabilities were reported by the completed scan?
4. What severity was assigned to the UnrealIRCd Backdoor Detection finding?
5. Why must vulnerability scanning be restricted to authorized targets?

subsectionPractical Lessons from the Nessus Assessment The practical demonstrated the value of moving from an overall scan view to individual plugin details. The results page gives the analyst a broad understanding of severity and volume, while plugin pages provide the evidence required for individual findings.

subsubsectionFrom Scan to Report A scanner report is only the starting point. The security analyst must interpret findings, select representative evidence, understand potential impact, identify remediation, and communicate the result clearly. This transformation from raw output to a structured finding is an important practical cybersecurity skill.

newpage
subsectionExtended Technical Analysis: Nessus Practical Assessment The Nessus practical connects the theoretical vulnerability assessment lifecycle with an actual assessment interface. The screenshots demonstrate the progression from deployment and plugin activity to scan configuration, completed results, and detailed vulnerability evidence.

subsubsectionAssessment Setup The Nessus setup process established the scanner in the local Kali Linux environment. The Events interface showed successful downloads of Nessus core components, license information, and plugins. This is useful evidence because the scanner depends on current assessment plugins to recognize known vulnerabilities.

subsubsectionScan Configuration The completed scan was named “Metasploitable Vulnerability Assessment” and used a Vulnerability scan type with an On Demand schedule. The target was an intentionally vulnerable laboratory system. This configuration demonstrates a controlled assessment rather than an unsolicited scan of an external system.

subsubsectionCompleted Assessment The results interface reported 72 vulnerabilities and displayed a severity distribution. The result list included Critical, High, Medium, and other categories. This high-level page is useful for understanding the breadth of the assessment before selecting individual findings for deeper review.

subsubsectionPlugin-Level Review The UnrealIRCd and Apache Tomcat AJP Connector findings were selected because their plugin pages provided clear descriptions, solutions, identifiers, risk metrics, and observed output. These details are the type of evidence required to transform scanner output into a security finding.

subsectionNessus Evidence Interpretation
begintable[H]
centering
small
begintabularx
textwidth>
bfseriesp3.4cmX
toprule Evidence Element Interpretation

midrule Scan Name Identifies the assessment performed against the laboratory environment.

Scan Type Shows that the assessment was configured as a vulnerability scan.

Finding Count Indicates the number of vulnerabilities reported by the completed scan.

Severity Provides the first-level classification used to organize findings.

Plugin ID Provides a unique reference to the Nessus detection logic.

Description Explains the security condition identified by the plugin.

Solution Provides remediation guidance associated with the finding.

Risk Metrics Adds technical and prioritization context such as CVSS, VPR and EPSS where available.

Output Shows evidence observed during the scan.

bottomrule
endtabularx
captionInterpretation of evidence available in the Nessus assessment
endtable

subsectionPractical Limitations The scan represents the state of the authorized laboratory target at the time it was assessed. Results can vary with target availability, service configuration, plugin coverage, network conditions, and scan settings. Therefore, the report documents the observed assessment rather than claiming that the same result applies universally to every environment.

subsectionTask 5 Learning Outcome The practical demonstrates how a security analyst moves from scanner operation to evidence-based reporting. The important skill is not simply running Nessus; it is interpreting the findings, selecting representative evidence, understanding severity and risk metrics, and writing actionable remediation recommendations.

newpage
subsectionCase Study: Interpreting the Nessus Results Page The Nessus results page provides a useful starting point for assessment review. The completed scan displayed the overall vulnerability count and grouped results according to severity. An analyst can begin by understanding the distribution, then select important findings for detailed inspection.

The plugin detail page adds another level of evidence. For the UnrealIRCd finding, the report shows the Critical severity, plugin ID, description, solution, affected service, and risk information. For the Apache Tomcat AJP Connector finding, the page provides the plugin description, solution, scoring information, and output associated with the vulnerable service.

The important analytical step is connecting the evidence to the report. The analyst should not simply copy every field. Instead, the analyst selects the information that explains what was detected, why it matters, and what should be done.

subsectionEvidence Selection Principles
beginitemize
item Use the scan overview to establish the assessment result.
item Use the plugin detail page to support individual findings.
item Record unique identifiers such as plugin IDs where relevant.

item Record severity and supporting risk metrics accurately.
item Use the solution provided by the tool as remediation evidence.
item Avoid exposing unnecessary sensitive information.
item Do not repeat the same screenshot when a distinct screenshot already supports the point.
enditemize

The practical therefore demonstrates both tool operation and security-analysis judgment.

5.11 Task 5 Summary

The practical demonstrated a controlled vulnerability assessment using Nessus Essentials against an intentionally vulnerable Metasploitable system. The completed scan produced 72 findings and provided severity, scoring, evidence, and remediation information for individual vulnerabilities.

6 Task 6: Security Finding Report

6.1 Introduction

A security finding report converts technical assessment results into information that can be understood and acted upon by system administrators, developers, security teams, and management. Each finding should clearly state the weakness, risk level, potential impact, and recommended remediation.

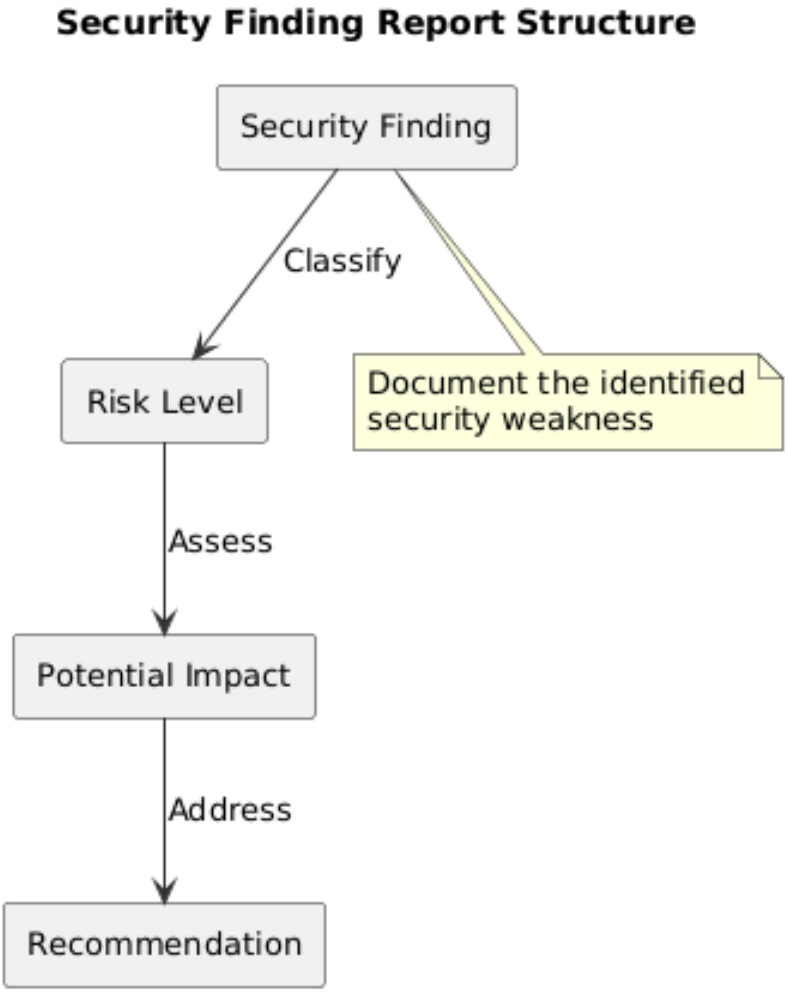


Figure 18: Structure of a security finding report

6.2 Finding 1 – UnrealIRCd Backdoor Detection

Finding	UnrealIRCd Backdoor Detection (Plugin 46882)
Risk Level	Critical
Possible Impact	Potential arbitrary code execution on the affected host through the vulnerable backdoored service.
Recommendation	Re-download and verify the software using trusted checksums, reinstall a clean supported version, and remove the affected backdoored component.
Evidence	Nessus identified the finding on the laboratory target and displayed the affected service on TCP port 6667.

Table 4: Security finding report – UnrealIRCd Backdoor Detection

6.3 Finding 2 – Apache Tomcat AJP Connector Request Injection

Finding	Apache Tomcat AJP Connector Request Injection (Ghostcat), Plugin 134862
Risk Level	Critical
Possible Impact	Potential unauthorized reading of web application files through a vulnerable AJP connector; impact can be greater where vulnerable configurations permit file upload.
Recommendation	Require authorization for the AJP connector and upgrade the affected Tomcat deployment to a supported version addressing the vulnerability.
Evidence	Nessus identified the finding and provided plugin-level description, scoring, output, and remediation information.

Table 5: Security finding report – Apache Tomcat AJP Connector Request Injection

6.4 How to Read a Nessus Finding

A Nessus finding can contain several technical fields. The finding name identifies the weakness, the plugin ID provides a unique scanner reference, severity provides an initial classification, and scoring fields such as CVSS, VPR, or EPSS provide additional risk context.

6.4.1 CVSS

CVSS provides a standardized way of expressing the technical severity of a vulnerability using characteristics such as attack vector, complexity, privileges, user interaction, scope, and impact.

6.4.2 VPR

Vulnerability Priority Rating is a Tenable risk-prioritization metric that can help determine which vulnerabilities deserve attention.

6.4.3 EPSS

Exploit Prediction Scoring System information estimates the likelihood of exploitation in the wild and can provide additional context for prioritization.

6.5 Finding Documentation Quality

subsectionEvidence Quality and Reproducibility A professional finding should contain enough information for another security professional to understand the observation without needing to guess what was assessed. Useful evidence includes the affected host, service, plugin identifier, severity, relevant output, and the reasoning behind the recommendation.

subsubsectionClear Language Findings should distinguish confirmed observations from possible consequences. Phrases such as “Nessus identified” or “the plugin reported” describe evidence, while phrases such as “could allow” describe potential impact. This distinction improves technical accuracy.

subsubsectionActionable Recommendations Recommendations should be specific enough to guide remediation. Instead of simply stating “fix the vulnerability,” the report should identify an appropriate corrective action such as upgrading a component, changing a configuration, removing an unnecessary service, or strengthening an authentication control.

Each reported finding should be understandable without requiring the reader to inspect the entire scanner interface.

6.5.1 Finding

State exactly what weakness was detected and identify the relevant component.

6.5.2 Risk Level

Record the severity observed in the assessment and explain any important contextual factors.

6.5.3 Potential Impact

Describe what an attacker could potentially achieve if the weakness were successfully abused.

6.5.4 Recommendation

Provide a practical corrective action such as upgrading software, changing configuration, disabling an unnecessary service, strengthening authentication, or applying an appropriate security control.

6.6 Remediation Priority

Critical findings generally receive immediate attention, but final priority should still consider exposure and asset context. High and Medium findings should be tracked through the organization's remediation process, while Low and Informational findings may still provide useful hardening opportunities.

6.7 Vulnerability Finding and Risk Prioritization

The assessment process should validate findings, evaluate likelihood and impact, assign a priority, and select an appropriate remediation action.

6.8 Security Risk Rating

Risk prioritization should consider both the likelihood that a weakness can be exploited and the potential impact on the affected system or organization. Severity ratings from scanning tools provide useful technical context but should be interpreted alongside asset criticality and exposure.

subsectionReport Quality Checklist Before a security finding is finalized, the assessor should confirm that the title is precise, the affected asset is identified, the evidence supports the finding, the severity is recorded correctly, the impact is realistic, and the recommendation is actionable.

subsubsectionVerification Status A finding should remain traceable after remediation. The report can record whether the issue is open, being remediated, accepted according to the appropriate process, or verified as fixed. This creates continuity between assessment, remediation, and re-testing.

newpage
subsectionExtended Technical Analysis: Security Finding Documentation A security finding report is the bridge between technical assessment and remediation. A scanner can identify a weakness, but system owners need a concise explanation of what was found, why it matters, and what should be done.

subsubsectionFinding Identification The finding title should be specific enough to identify the weakness and affected component. Where available, a plugin identifier or other unique reference improves traceability.

subsubsectionRisk Description The report should record the observed severity and relevant scoring information. Technical metrics such as CVSS can support the explanation, while additional context such as exposure and asset criticality can influence final priority.

subsubsectionPotential Impact Impact statements should be realistic and based on the evidence available. A finding should not claim that a system was fully compromised unless the assessment actually demonstrated that outcome. Potential consequences should be clearly identified as potential.

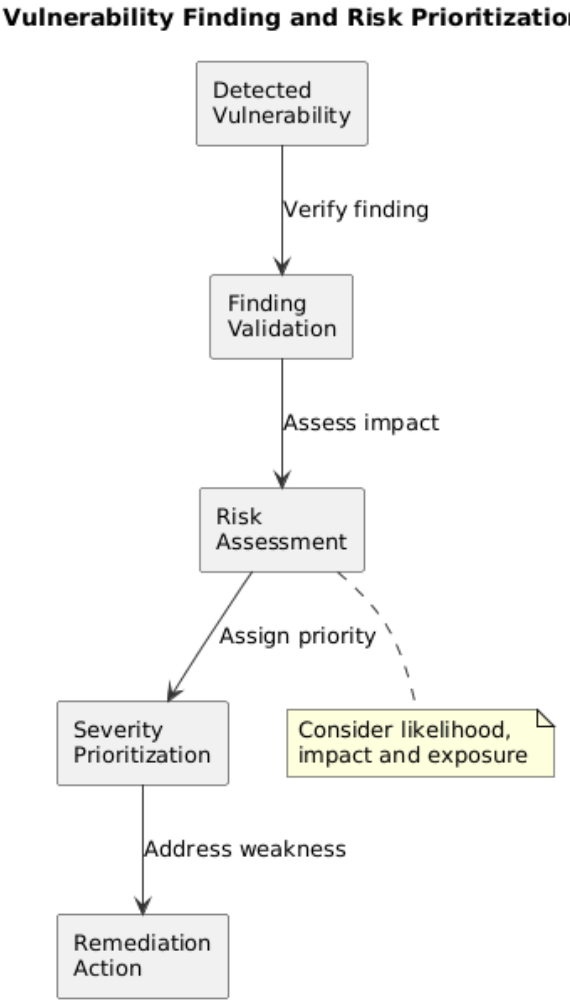


Figure 19: Vulnerability finding analysis and risk prioritization workflow



Figure 20: Security risk evaluation based on likelihood and potential impact

subsubsectionRecommendation Recommendations should address the root cause. For example, an outdated component should normally be upgraded or replaced, while an unnecessary exposed service may need to be disabled or restricted. A good recommendation gives the system owner a clear next action.

subsubsectionEvidence Evidence can include scanner output, plugin details, affected ports, version information, configuration observations, and screenshots. The evidence should support the finding while avoiding unnecessary exposure of sensitive information.

subsectionExample Finding Review Workflow
beginenumerate
item Confirm that the finding belongs to the authorized assessment.
item Identify the affected asset and service.
item Review the plugin or technical evidence.
item Record severity and relevant risk metrics.
item Determine realistic potential impact.
item Identify the recommended remediation.
item Assign an appropriate remediation priority.
item Verify the issue after corrective action.
endenumerate

subsectionManagement and Technical Audiences A useful report should serve different readers. Technical teams need enough detail to reproduce and remediate the issue, while management needs a clear explanation of business significance and priority. Structured tables and concise evidence help the same finding serve both audiences.

subsectionTask 6 Learning Outcome Task 6 demonstrates how raw vulnerability data becomes a professional security deliverable. A well-written finding is precise, evidence-based, actionable, and traceable through remediation and verification.

newpage
subsectionCase Study: Converting Scanner Output into a Finding A scanner may produce dozens of technical observations, but a professional report should focus attention on findings that are relevant and actionable. The first step is to identify the affected asset and the exact security condition. The second is to review the evidence and confirm the finding. The third is to explain severity and possible impact without overstating what was actually demonstrated.

For the UnrealIRCd finding, the report identifies the plugin, Critical severity, affected IRC service, potential arbitrary code-execution consequence, and the remediation guidance shown by Nessus. For the Ghostcat finding, the report identifies the vulnerable AJP connector, Critical severity, possible unauthorized file access, and the recommended configuration or upgrade action.

These examples show why a finding should be written as a complete security story: observation, evidence, risk, impact, recommendation, and verification. The final reader should be able to understand the issue and decide what action is required.

subsectionFinding Quality Test A finding is stronger when another security professional can answer the following questions from the report itself:
beginenumerate
item What was found?
item Where was it found?
item What evidence supports it?
item How severe is it?
item What could happen?
item What should be done?

item How will the correction be verified?
endenumerate

6.9 Task 6 Summary

A useful security finding report connects each weakness to its risk, possible impact, evidence, and remediation. The practical findings from Nessus demonstrate how scanner output can be converted into actionable security recommendations.

6.10 Final Week 4 Summary

This Week 4 assignment provided a structured introduction to Vulnerability Assessment and Penetration Testing. The report covered the concepts of vulnerabilities, threats, risks, exploits, common security weaknesses, the vulnerability assessment lifecycle, penetration testing methodology, authorization requirements, and safe practical assessment.

The practical activity demonstrated the use of Nessus Essentials against an intentionally vulnerable Metasploitable laboratory system. The completed scan identified 72 vulnerabilities, including Critical findings such as UnrealIRCd Backdoor Detection and Apache Tomcat AJP Connector Request Injection. These results were documented with risk, impact, evidence, and remediation recommendations.

The overall learning outcome is that professional cybersecurity assessment is not simply about finding vulnerabilities. It requires authorized scope, systematic assessment, careful analysis, risk-based prioritization, safe testing, evidence collection, remediation, and verification.

7 VAPT Assessment Checklist and Reporting Notes

A repeatable checklist helps ensure that an assessment is performed consistently. The following checklist summarizes the practical activities covered in this assignment and can be reused for future authorized laboratory assessments.

No.	Checkpoint	Expected Evidence
1	Confirm authorization	Defined target, owner, scope and testing conditions.
2	Identify assets	Hostnames, addresses, services and application context.
3	Configure assessment	Appropriate scan type, target and timing.
4	Execute scan	Completed assessment with recorded status.
5	Review findings	Severity, affected assets and supporting evidence.
6	Validate priority findings	Plugin details, configuration evidence or manual confirmation where appropriate.
7	Recommend remediation	Specific corrective action and responsible owner.
8	Verify remediation	Re-scan or re-test evidence.

Table 6: Reusable vulnerability assessment checklist

7.1 Evidence Handling Notes

Screenshots are useful because they provide a visual record of the assessment environment. However, screenshots should complement rather than replace written evidence. A good report explains what the reader is seeing, identifies the relevant finding, and connects the evidence to the recommendation.

7.2 Professional Reporting Language

Security reports should be precise, neutral, and evidence-based. Avoid unnecessary alarmist language. When the evidence demonstrates a scanner detection, describe the detection accurately and then explain the potential consequence separately. This approach makes the report useful to both technical and non-technical readers.

7.3 Final Learning Reflection

The Week 4 activities connect foundational security terminology with a practical vulnerability assessment. The progression from vulnerability concepts to common weaknesses, assessment lifecycle, penetration testing, Nessus scanning, and finding documentation demonstrates how security professionals move from identifying a technical weakness to communicating a remediation decision.

8 Appendix A: Vulnerability Assessment Working Notes

This appendix provides a reusable analytical framework for future authorized assessments. It is included as a practical learning aid and follows the concepts demonstrated in the main tasks.

8.1 Asset Review Questions

- What system or application is being assessed?
- What is the business purpose of the asset?
- Who owns or maintains it?
- Is it Internet-facing, internal, or isolated?
- Which services and ports are intentionally exposed?
- What data or functions are considered sensitive?
- What security controls already protect the asset?

8.2 Finding Review Questions

- Is the finding relevant to the assessed asset?
- What evidence supports the detection?
- What component or configuration causes the weakness?
- What conditions are required for exploitation?
- What privileges could be obtained?
- What is the realistic potential impact?
- What is the most appropriate remediation?
- How will remediation be verified?

8.3 Severity Interpretation

Severity categories are useful for organizing findings, but the assessor should not rely on severity alone. A Critical finding on an isolated training host may be less urgent operationally than a High finding on a business-critical public-facing system. Final priority should therefore incorporate context.

8.4 False Positive Review

A scanner finding should be reviewed when the result appears inconsistent with the observed system. Validation may involve reviewing the installed version, service configuration, plugin output, or another authorized source of evidence. The goal is to improve the accuracy of the assessment rather than simply increase the number of findings reported.

8.5 False Negative Awareness

No vulnerability scanner detects every possible weakness. Detection can depend on plugin coverage, authentication, network reachability, application behavior, and configuration. Manual review and complementary testing may therefore be required for higher assurance.

9 Appendix B: Professional VAPT Report Checklist

Report Element	Professional Requirement
Title and Scope	Clearly identify the assessment and its authorized boundary.
Methodology	Explain the approach and major assessment stages.
Target Information	Identify the assessed laboratory system without unnecessary sensitive information.
Finding Title	Use a precise and recognizable vulnerability name.
Evidence	Include supporting observations, plugin information or screenshots.
Severity	Record the observed severity and relevant scoring information.
Impact	Explain realistic technical or business consequences.
Recommendation	Provide an actionable corrective measure.
Status	Track remediation and verification state.
Safety Statement	Confirm that testing was restricted to authorized systems.

Table 7: Professional VAPT report checklist

9.1 Evidence Presentation Principles

Evidence should be readable, relevant, and positioned close to the discussion it supports. Screenshots should not be repeated merely to increase report length. Each image should have a clear caption and should contribute a distinct piece of evidence or explanation.

9.2 Writing Principles

Professional cybersecurity writing should be precise and neutral. Avoid unsupported claims, distinguish observation from potential impact, define technical terms before using them extensively, and explain why a recommendation reduces the identified risk.

9.3 Closing Reflection

The complete VAPT workflow can be summarized as: understand the weakness, establish authorization, identify the target, assess it systematically, interpret the evidence, prioritize the risk, recommend remediation, and verify the result. This workflow forms a foundation for more advanced security operations, vulnerability management, penetration testing, and SOC activities.

10 Appendix C: VAPT Terminology and Practical Notes

The following terminology reference consolidates important terms used throughout the assignment. It is intended as a quick revision section for future security assessments.

Term	Practical Meaning
Asset	A system, application, service, device, identity, or information resource that has value.
Attack Surface	The collection of reachable or exposed points through which a system may be interacted with.
Vulnerability	A weakness or condition that can reduce the security of an asset.
Threat	A potential source, event, or circumstance capable of causing harm.
Exploit	A technique or method used to take advantage of a vulnerability.
Impact	The potential consequence of a successful security event.
Risk	Potential loss or harm considering likelihood and impact.
Vulnerability Assessment	A systematic process of identifying, analyzing, prioritizing, and reporting weaknesses.
Penetration Testing	An authorized security assessment that validates selected weaknesses and evaluates potential impact.
Reconnaissance	Authorized information gathering used to understand a target environment.
Scanning	Automated or manual examination used to identify services, versions, configurations, or possible weaknesses.
Enumeration	Detailed identification of resources, services, or information exposed by a target.
Remediation	Corrective action taken to remove or reduce a security weakness.
Re-scan	A subsequent assessment used to verify whether a previously identified issue remains.
Severity	A technical classification indicating the seriousness of a vulnerability.
CVSS	Common Vulnerability Scoring System used to express technical vulnerability severity.
VPR	Vulnerability Priority Rating, a Tenable prioritization metric.
EPSS	Exploit Prediction Scoring System information used to estimate exploitation likelihood.
Least Privilege	The principle of granting only the access required for an approved task.
Defense in Depth	The use of multiple complementary controls so one failed control does not result in complete compromise.

10.1 Why Consistent Terminology Matters

Security reports are often read by people with different technical backgrounds. Consistent terminology reduces ambiguity and makes findings easier to compare across assessments. It also helps ensure that a vulnerability is not incorrectly described as a confirmed compromise when the evidence only demonstrates a potential weakness.

11 Appendix D: Vulnerability Assessment Review Questions

The following questions can be used as a revision exercise after completing the assignment.

11.1 Foundational Concepts

1. Explain the difference between a vulnerability and a threat.
2. Why is an exploit different from a vulnerability?
3. Why can the same vulnerability create different levels of risk on different assets?
4. How do likelihood and impact influence risk?
5. Why should a vulnerability scanner result be validated?

11.2 Common Vulnerabilities

1. Why are parameterized queries important for SQL Injection prevention?
2. Why does XSS prevention depend on output context?
3. How can session management weaknesses affect authentication security?
4. Give examples of security misconfiguration.
5. Why does dependency inventory matter for vulnerable components?

11.3 Assessment Process

1. What should be established before scanning begins?
2. What is the purpose of analysis after scanning?
3. Which factors can influence remediation priority?
4. Why should remediation be verified?
5. How can repeated findings reveal weaknesses in security processes?

11.4 Penetration Testing

1. Why is written permission required?
2. What is the purpose of the rules of engagement?
3. What is the difference between reconnaissance and scanning?
4. Why should controlled testing use the least intrusive effective method?
5. What information should a professional penetration-test report contain?

11.5 Nessus Practical

1. What was the authorized target used in the practical?
2. What scan type was used?
3. How many vulnerabilities were reported?
4. What information is available on a Nessus plugin detail page?
5. Why should severity be interpreted together with context?

12 Appendix E: Final VAPT Workflow Reference

The complete workflow demonstrated in this assignment can be summarized as a continuous security-improvement cycle.

1. **Understand:** Identify the security concept, weakness, threat, impact, and risk.
2. **Authorize:** Establish written permission, scope, targets, constraints, and contacts.
3. **Identify:** Build an accurate picture of the assets and services inside the approved scope.
4. **Assess:** Use an appropriate vulnerability assessment method to identify possible weaknesses.
5. **Analyze:** Validate findings and understand their technical and environmental context.
6. **Prioritize:** Rank findings using severity, exploitability, exposure, asset criticality, likelihood, and impact.
7. **Remediate:** Apply patches, configuration changes, access controls, upgrades, or compensating controls.
8. **Verify:** Re-scan or re-test to confirm that the weakness has been addressed.
9. **Report:** Communicate evidence, risk, impact, and remediation clearly.
10. **Improve:** Use repeated findings and assessment metrics to strengthen security processes.

12.1 Professional Skills Demonstrated

This assignment required more than knowledge of security terminology. It involved understanding assessment scope, interpreting vulnerability evidence, using a vulnerability scanner, distinguishing technical severity from overall risk, documenting practical findings, and communicating remediation recommendations.

12.2 Connection to SOC and Security Operations

Vulnerability assessment supports security operations because weaknesses identified during assessment can become useful inputs for monitoring and defensive planning. A security team can use vulnerability information to understand which services require closer monitoring, which systems need stronger controls, and which weaknesses could increase the significance of suspicious activity.

12.3 Connection to Future Penetration Testing

The concepts introduced here form a foundation for more advanced penetration-testing work. Before using advanced testing techniques, a security professional should understand networking, operating systems, authentication, web technologies, vulnerability assessment, authorization, evidence handling, and responsible disclosure.

12.4 Final Reflection

The Week 4 assignment demonstrates a complete progression from fundamental vulnerability concepts to practical assessment and professional reporting. The Nessus laboratory activity provided direct experience with scan configuration, result interpretation, severity analysis, plugin evidence, and remediation guidance. Together, the tasks establish a practical foundation for future work in vulnerability management, penetration testing, SOC operations, and defensive security.

13 Appendix F: Quick Reference for Future Assessments

Stage	Key Question
Authorization	Am I explicitly permitted to assess this target?
Scope	Which systems, services and dates are approved?
Identification	What assets and services exist inside the scope?
Scanning	What weaknesses and technical indicators were detected?
Analysis	Does the evidence support the finding?
Prioritization	Which findings require the fastest response and why?
Remediation	What corrective action addresses the root cause?
Verification	Did the corrective action actually remove or reduce the weakness?
Reporting	Can another reader understand the finding and act on it?

Table 9: Quick reference for an authorized vulnerability assessment

13.1 Final Checklist

Before submitting an assessment report, confirm that the scope is clear, every major finding has evidence, severity and impact are explained, recommendations are actionable, screenshots are relevant and unique, references are accessible through the report's blue "[View Reference]" links, and the final document has been reviewed for formatting and alignment.